



ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับ
การใช้งานภาษาไทยและอังกฤษแบบผสม

โดย

นายชิตพัทธ์ จำปา

โครงการพิเศษนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิทยาศาสตร์บัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2569
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับ
การใช้งานภาษาไทยและอังกฤษแบบผสม

โดย

นายชิตพัทธ์ จำปา

โครงการพิเศษนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิทยาศาสตร์บัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2569
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

**AI-Generated Text Detection and Classification System for Thai-
English Code-switching**

BY

Mr. Chitipat Jampa

**A FINAL-YEAR PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF SCIENCE
COMPUTER SCIENCE
FACULTY OF SCIENCE AND TECHNOLOGY
THAMMASAT UNIVERSITY
ACADEMIC YEAR 2026
COPYRIGHT OF THAMMASAT UNIVERSITY**

มหาวิทยาลัยธรรมศาสตร์
คณะวิทยาศาสตร์และเทคโนโลยี

รายงานโครงการพิเศษ

ของ

นายชิตพัทธ์ จำปา

เรื่อง

ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับการใช้งานภาษาไทยและ
อังกฤษแบบผสม

ได้รับการตรวจสอบและอนุมัติ ให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
เมื่อ วันที่ วันที่ ชื่อเต็มของเดือน พ.ศ. 2569

อาจารย์ที่ปรึกษา

(ผศ.ดร.ปกรณ์ ลีสุทธิพรชัย)

กรรมการสอบโครงการพิเศษ

(ผศ.ดร.วิลาวรรณ รักผกาวงศ์)

กรรมการสอบโครงการพิเศษ

(อาจารย์สิริกัญญา นิลพานิช)

หัวข้อโครงการพิเศษ

ระบบตรวจจับและจำแนกข้อความที่สร้างโดย
ปัญญาประดิษฐ์สำหรับการใช้งานภาษาไทยและ
อังกฤษแบบผสม

ชื่อผู้เขียน

นายชิตีพัทธ์ จำปา

ชื่อปริญญา

วิทยาศาสตร์บัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์

สาขาวิชา/คณะ/มหาวิทยาลัย

สาขาวิชาวิทยาการคอมพิวเตอร์

คณะวิทยาศาสตร์และเทคโนโลยี

มหาวิทยาลัยธรรมศาสตร์

อาจารย์ที่ปรึกษาโครงการพิเศษ

ผศ.ดร.ปกรณ์ ลีสุทธิพรชัย

ปีการศึกษา

2569

บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนาต้นแบบระบบตรวจสอบข้อความที่สร้างโดยปัญญาประดิษฐ์ (AI-generated text) โดยมุ่งเน้นการแก้ปัญหาความซับซ้อนของการใช้ภาษาไทยร่วมกับภาษาอังกฤษ (Thai-English Code-switching) และการรักษาความเป็นส่วนตัวของผู้ใช้งาน ซึ่งเป็นข้อจำกัดสำคัญของเครื่องมือในปัจจุบันที่มักอาศัยการส่งข้อมูลไปประมวลผลบนคลาวด์เซิร์ฟเวอร์ (Server-side)

ในส่วนของวิธีการดำเนินงาน ผู้จัดทำออกแบบการพัฒนาโดยใช้แบบจำลองภาษา "WangchanBERTa" เป็นโครงสร้างหลักในการจำแนกข้อความ โดยจะทำการปรับแต่ง (Fine-tuning) ด้วยชุดข้อมูลที่คัดสรรมาเพื่อรองรับบริบทภาษาผสม จุดเด่นของโครงการนี้คือการออกแบบสถาปัตยกรรมแบบ "Client-side Intelligence" ซึ่งจะมีการแปลงแบบจำลองให้อยู่ในรูปแบบ ONNX และใช้เทคนิคการบีบอัด (Model Quantization) เพื่อให้สามารถรันการประมวลผลได้โดยตรงบนเว็บเบราว์เซอร์ของผู้ใช้

ผลลัพธ์ที่คาดหวังจากโครงการนี้คือ ระบบที่สามารถระบุแหล่งที่มาของข้อความระหว่างมนุษย์และปัญญาประดิษฐ์ได้อย่างแม่นยำบนอุปกรณ์ปลายทาง โดยที่ข้อความของผู้ใช้จะไม่ถูกส่งออกจากเครื่องเพื่อรักษาความเป็นส่วนตัว นอกจากนี้ ระบบยังถูกออกแบบให้รองรับการรับข้อมูลป้อนกลับจากผู้ใช้งาน (Feedback Loop) เพื่อนำมาพัฒนาความสามารถของแบบจำลองให้ทันสมัยอย่างต่อเนื่องในอนาคต

คำสำคัญ: การตรวจจับข้อความปัญญาประดิษฐ์, ภาษาไทยปนอังกฤษ, WangchanBERTa, การประมวลผลฝั่งผู้ใช้งาน, ความเป็นส่วนตัวของข้อมูล

Thesis Title	AI-Generated Text Detection and Classification System for Thai-English Code-switching
Author	Mr. Chitipat Jampa
Degree	Bachelor of Science
Major Field/Faculty/University	Computer Science Faculty of Science and Technology Thammasat University
Project Advisor	Asst.Prof.Dr. Pakorn Leesutthipornchai
Academic Years	2026

ABSTRACT

This project proposes the development of a detection system for AI-generated text, specifically targeting the challenges of Thai-English code-switching and data privacy. Traditional detection methods often rely on server-side processing, which poses potential risks to user confidentiality and struggles with mixed-language contexts.

The proposed methodology utilizes the "WangchanBERTa" model as the core architecture for text classification. The model will be fine-tuned with a specialized dataset designed to handle code-switching environments. A key innovation of this project is the "Client-side Intelligence" architecture, where the model is converted to the ONNX format and optimized through quantization to enable direct inference within the user's web browser.

The expected outcome is a high-performance system capable of distinguishing between human-written and AI-generated content locally on the user's device. This approach ensures zero-data leakage, as text processing remains entirely within the client environment. Furthermore, the system includes a feedback mechanism to allow for offline retraining and continuous model improvement.

Keywords: AI-Generated Text Detection, Thai-English Code-switching, WangchanBERTa, Client-side Processing, Data Privacy

กิตติกรรมประกาศ

โครงการฉบับนี้สำเร็จลุล่วงไปได้ด้วยความกรุณาจากอาจารย์ปกรณ์ ลีสุทธิพรชัย อาจารย์ที่ปรึกษาโครงการ ที่ได้สละเวลาให้คำปรึกษา แนะนำแนวทาง และถ่ายทอดความรู้ที่เป็นประโยชน์อย่างยิ่งในการวางรากฐานโครงการ ตลอดจนการเตรียมชุดข้อมูลและการวิเคราะห์ในเบื้องต้น ผู้วิจัยจึงขอขอบพระคุณมา ณ โอกาสนี้

นายชิตีพัทธ์ จำปา

สารบัญ

	หน้า
บทคัดย่อ	2
ABSTRACT	4
กิตติกรรมประกาศ	5
สารบัญ	6
สารบัญตาราง	9
สารบัญภาพ	10
รายการสัญลักษณ์และคำย่อ	11
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและความสำคัญของโครงการ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของโครงการ	2
1.4 ประโยชน์ของโครงการ	3
1.5 ข้อจำกัดของโครงการ	4
บทที่ 2 วรรณกรรมและงานวิจัยที่เกี่ยวข้อง	6
2.1 แนวคิดทฤษฎีที่เกี่ยวข้อง	6
2.1.1 การจัดการภาษาไทยด้วยโมเดล WangchanBERTa	6
2.1.2 หลักการตรวจจับข้อความที่สร้างโดยปัญญาประดิษฐ์	7
2.1.3 การประมวลผลฝั่งผู้ใช้งาน (Client-side Intelligence) และความเป็นส่วนตัวของข้อมูล	8

2.1.4	การบีบอัดแบบจำลอง (Model Quantization) และเทคโนโลยี ONNX Runtime	
Web	8	
2.2	งานวิจัยที่เกี่ยวข้อง	9
2.2.1	การวิเคราะห์ประสิทธิภาพและข้อจำกัดของระบบตรวจจับในปัจจุบัน	9
บทที่ 3	วิธีการวิจัย	11
3.1	ภาพรวมของโครงการ	11
3.1.1	ขั้นตอนการพัฒนาแบบจำลอง (Offline Development Phase)	12
3.1.2	ขั้นตอนการบีบอัดและแปลงรูปแบบ (Model Optimization Phase)	12
3.1.3	ขั้นตอนการอนุมานผลบนเบราว์เซอร์ (Client-side Inference Phase)	12
3.2	การวิเคราะห์ขอบเขตและความต้องการของระบบ	12
3.2.1	Flow Chart	13
3.2.2	ความต้องการด้านฟังก์ชันการทำงาน (Functional Requirements)	13
3.2.3	ความต้องการด้านคุณภาพและความปลอดภัย (Non-functional Requirements)	14
3.2.4	ความต้องการด้านสภาพแวดล้อมทางเทคนิค (Technical Requirements)	14
3.2.5	การรวบรวมและจัดเตรียมชุดข้อมูล (Data Preparation)	14
3.3	ประเด็นที่น่าสนใจและสิ่งที่ท้าทาย	17
3.3.1	การรักษาสมดุลระหว่างขนาดและความแม่นยำ (Size vs. Accuracy Trade-off)	17
3.3.2	ความซับซ้อนของภาษาผสม (Tokenization Challenges)	17
3.4	ผลลัพธ์ที่คาดหวัง	17
3.5	ระบบต้นแบบและผลลัพธ์เบื้องต้น	17
3.5.1	ตัวอย่างภาพรวมของระบบ	18
3.5.2	หน้าแสดงผลตัวอย่างการจำแนกข้อความ	20
3.5.3	ระบบรับข้อมูลป้อนกลับและการรายงานผล	22

บทที่ 4	ทรัพยากรและแผนการดำเนินงาน	25
4.1	การจัดเตรียมฮาร์ดแวร์และซอฟต์แวร์	25
4.1.1	ฮาร์ดแวร์ (Hardware)	25
4.1.2	ซอฟต์แวร์ (software and tool)	25
4.2	แผนการดำเนินงาน	26
บทที่ 5	สรุป	28
5.1	สรุปผลการดำเนินงาน	28
5.2	อภิปรายผลและข้อจำกัด	28
5.3	ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต	29
	รายการอ้างอิง	30
	ภาคผนวก	31
	ภาคผนวก ก สคริปต์การรวบรวมและทำความสะอาดข้อมูล	32
	ก.1 สคริปต์รวบรวมข้อมูลจาก Pantip.com	32
	ก.2 สคริปต์ทำความสะอาดข้อมูล	35

สารบัญตาราง

	หน้า
ตารางที่ 2.1 เปรียบเทียบและเหตุผลการเลือกโมเดล	7
ตารางที่ 2.2 การวิเคราะห์ช่องว่างทางงานวิจัย	10
ตารางที่ 3.1 สรุปรายละเอียดสถิติของชุดข้อมูล	15
ตารางที่ 3.2 ตัวอย่างการประมวลผลข้อมูลเพื่อขจัดสิ่งเจือปน (Data Cleaning Examples)	16
ตารางที่ 4.1 การดำเนินงานที่ผ่านมาจนถึงปัจจุบัน	26
ตารางที่ 4.2 แผนการดำเนินงานในอนาคต	27

สารบัญภาพ

หน้า

ภาพที่ 2.1 การเปรียบเทียบสถาปัตยกรรมพื้นฐานของโมเดลภาษาขนาดใหญ่ (ที่มา: VISAI, ม.ป.ป.)	6
ภาพที่ 2.2 แผนภาพเปรียบเทียบสถาปัตยกรรมการอนุมานผลโมเดลปัญญาประดิษฐ์ระหว่างฟังก์คลาวด์เซิร์ฟเวอร์และฝั่งผู้ใช้งาน	8
ภาพที่ 2.3 กระบวนการแปลงค่าน้ำหนักของแบบจำลองจากรูปแบบทศนิยม 32 บิต (FP32) เป็นจำนวนเต็ม 8 บิต (INT8) ผ่านเทคนิค Quantization (ที่มา: GeeksforGeeks, ม.ป.ป.)	9
ภาพที่ 3.1 สถาปัตยกรรมภาพรวมของระบบตรวจจับข้อความปัญญาประดิษฐ์	11
ภาพที่ 3.2 แผนภาพกระบวนการเตรียมข้อมูลและการพัฒนาโมเดลตรวจจับข้อความ	13
ภาพที่ 3.3 แสดงหน้าต่างขอความยินยอมในการจัดการข้อมูล (Privacy Consensus) ในโหมดสว่างและโหมดมืด	18
ภาพที่ 3.4 แสดงหน้าหลักของระบบ (Homepage) ในโหมดสว่างและโหมดมืด	19
ภาพที่ 3.5 แสดงผลการทดสอบข้อความที่สร้างโดยปัญญาประดิษฐ์ (AI Text) ในโหมดสว่างและโหมดมืด	20
ภาพที่ 3.6 แสดงผลการทดสอบข้อความที่เขียนโดยมนุษย์ (Human Text) ในโหมดสว่างและโหมดมืด	21
ภาพที่ 3.7 แสดงหน้าเว็บการกดปุ่มรายงานผล (Press report button) ในโหมดสว่างและโหมดมืด	22
ภาพที่ 3.8 แสดงหน้าเว็บหลังจากกดปุ่มรายงานผล (After press report button) ในโหมดสว่างและโหมดมืด	23
ภาพที่ 3.9 แสดงการแจ้งเตือนเมื่อดำเนินการส่งข้อมูลสำเร็จ (Thank you message) ในโหมดสว่างและโหมดมืด	24

รายการสัญลักษณ์และคำย่อ

สัญลักษณ์/คำย่อ	คำเต็ม/คำจำกัดความ
AI	Artificial Intelligence / ปัญญาประดิษฐ์
BBPE	Byte-level Byte Pair Encoding
CS	Code-switching
FP32	32-bit Floating Point
INT8	8-bit Integer
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
PDPA	Personal Data Protection Act
UI	User Interface
UX	User Experience

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของโครงการ

ในปัจจุบัน เทคโนโลยีปัญญาประดิษฐ์ประเภทโมเดลภาษาขนาดใหญ่ (Large Language Models: LLMs) เช่น ChatGPT และ Gemini ได้เข้ามามีบทบาทอย่างแพร่หลายในการสร้างสรรค์เนื้อหาและช่วยในการทำงานด้านภาษา อย่างไรก็ตาม ความสามารถในการสร้างข้อความที่ใกล้เคียงกับมนุษย์นั้นนำมาซึ่งความท้าทายในการตรวจสอบแหล่งที่มาของข้อมูล โดยเฉพาะในบริบทของประเทศไทยที่มีลักษณะการสื่อสารแบบสลับภาษาไทยปนอังกฤษ (Thai-English Code-switching) ซึ่งเป็นอุปสรรคสำคัญที่ทำให้เครื่องมือตรวจจับในปัจจุบันยังมีประสิทธิภาพที่ไม่แม่นยำเพียงพอ

นอกจากประเด็นด้านความแม่นยำแล้ว ระบบตรวจจับส่วนใหญ่ในปัจจุบันมักอาศัยการประมวลผลบนคลาวด์เซิร์ฟเวอร์ (Server-side) ซึ่งจำเป็นต้องส่งข้อความของผู้ใช้ไปยังระบบภายนอก กระบวนการนี้ก่อให้เกิดความกังวลด้านความเป็นส่วนตัว (Data Privacy) และข้อจำกัดในการใช้งานที่ต้องพึ่งพาสัญญาณอินเทอร์เน็ตและความพร้อมของเซิร์ฟเวอร์ส่วนกลางอยู่ตลอดเวลา

ด้วยเหตุนี้ ผู้จัดทำจึงมุ่งพัฒนา ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับการใช้งานภาษาไทยและอังกฤษแบบผสม โดยเน้นสถาปัตยกรรมที่ประมวลผลบนอุปกรณ์ของผู้ใช้งานโดยตรง (Client-side Intelligence) ผ่านการปรับแต่งโมเดล WangchanBERTa และใช้เทคนิคการบีบอัดโมเดลเพื่อการทำงานบนเว็บเบราว์เซอร์ เพื่อให้เป็นเครื่องมือที่ช่วยรักษาความโปร่งใสของข้อมูลควบคู่ไปกับการปกป้องความเป็นส่วนตัวของผู้ใช้งาน

1.2 วัตถุประสงค์

1. พัฒนาแบบจำลองต้นแบบในการจำแนกข้อความโดยใช้โครงสร้าง WangchanBERTa ภายใต้เงื่อนไขการทำงานแบบประมวลผลฝั่งผู้ใช้งาน (Client-side Processing) ที่ไม่มีการส่งข้อมูลออกสู่ภายนอก และรักษาความแม่นยำในการวิเคราะห์บริบทภาษาผสม
2. เสนอวิธีการบีบอัดแบบจำลอง (Model Quantization) และการแปลงรูปแบบไฟล์ (Format Conversion) เพื่อเพิ่มขีดความสามารถในการทำงานบนเว็บเบราว์เซอร์ที่มีทรัพยากรจำกัดได้อย่างมีประสิทธิภาพ

3. ทดสอบระบบกับชุดข้อมูลกรณีศึกษาที่ครอบคลุมข้อความภาษาไทยผสมอังกฤษ โดยมุ่งเน้นที่ข้อความจากแบบจำลองภาษาขนาดใหญ่ที่ได้รับความนิยมสูง (เช่น Llama 3) เป็นหลัก
4. เสนอแนวทางในการประยุกต์ใช้ระบบต้นแบบที่พัฒนาขึ้น สำหรับเป็นเครื่องมือรักษาความโปร่งใสของเนื้อหาดิจิทัลและปกป้องความเป็นส่วนตัวของข้อมูลภายในมหาวิทยาลัยธรรมศาสตร์

1.3 ขอบเขตของโครงการ

เพื่อให้การดำเนินงานเป็นไปตามวัตถุประสงค์ที่กำหนดไว้ ผู้จัดทำได้กำหนดขอบเขตของโครงการไว้ดังนี้:

1. ด้านโมเดลและการตรวจจับ (Model & Detection):

- ใช้แบบจำลองภาษา WangchanBERTa เป็นโครงสร้างหลักในการวิเคราะห์และจำแนกข้อความ
- จำแนกแหล่งที่มาของข้อความออกเป็น 2 ประเภท ได้แก่ ข้อความที่เขียนโดยมนุษย์, ข้อความที่สร้างโดยปัญญาประดิษฐ์
- รองรับการประมวลผลข้อความภาษาไทย และข้อความภาษาไทยที่มีการสลับภาษาอังกฤษ (Thai-English Code-switching)
- มุ่งเน้นการศึกษาข้อความที่สร้างจากปัญญาประดิษฐ์กรณีศึกษา คือ Llama 3 เนื่องจากเป็นโมเดลที่เลียนแบบการเขียนของมนุษย์ได้ใกล้เคียงความจริงในปัจจุบัน

2. ด้านสถาปัตยกรรมและการประมวลผล (Architecture & Processing):

- ระบบทำงานในรูปแบบเว็บแอปพลิเคชัน (Web Application) ที่ประมวลผลบนเบราว์เซอร์ของผู้ใช้งาน (Client-side) เท่านั้น
- ใช้เทคโนโลยี ONNX Runtime Web ในการรันโมเดลที่ผ่านการบีบอัด (Quantization) เพื่อให้ทำงานได้บนอุปกรณ์ที่มีทรัพยากรจำกัด
- ไม่มีการส่งข้อความของผู้ใช้ไปบันทึกหรือประมวลผลบนเซิร์ฟเวอร์ภายนอกในระหว่างขั้นตอนการประมวลผล (Inference Phase) เพื่อรักษาความเป็นส่วนตัวของข้อมูล (Zero-Data Leakage)

3. ด้านฟังก์ชันการทำงาน (Features):

- รองรับการนำเข้าข้อมูลผ่านการพิมพ์ข้อความโดยตรงบนหน้าเว็บ
- รองรับการดึงข้อความจากรูปภาพ (OCR) โดยใช้ไลบรารี Tesseract.js เพื่อนำมาตรวจสอบความเป็น AI
- มีระบบรับข้อมูลป้อนกลับ (Feedback Mechanism) ในกรณีที่ผู้ใช้ต้องการแจ้งผลการตรวจสอบที่ผิดพลาดกลับมายังผู้พัฒนา

4. ด้านข้อจำกัดของระบบ (Constraints):

- ระบบกำหนดเกณฑ์ขั้นต่ำในการนำเข้าข้อมูล โดยข้อความต้องมีความยาวอย่างน้อย 200 ตัวอักษร เพื่อให้แบบจำลองมีข้อมูลบริบทที่เพียงพอต่อการวิเคราะห์และจำแนกได้อย่างแม่นยำ
- ประสิทธิภาพในการตรวจจับขึ้นอยู่กับขีดความสามารถของโมเดล WangchanBERTa ที่ผ่านการ Fine-tuning แล้ว
- ความเร็วในการประมวลผลขึ้นอยู่กับทรัพยากร (Hardware) ของอุปกรณ์ที่ผู้ใช้ใช้เข้าถึงเบราว์เซอร์

1.4 ประโยชน์ของโครงการ

จากการดำเนินโครงการพัฒนาระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์ ผู้จัดทำคาดว่าจะได้รับประโยชน์ดังต่อไปนี้:

1. ด้านเครื่องมือและการตรวจสอบ (Tools & Verification):

- ได้ต้นแบบระบบที่สามารถจำแนกแหล่งที่มาของข้อความภาษาไทยปนอังกฤษได้อย่างแม่นยำ เพื่อใช้เป็นเครื่องมือตรวจสอบความโปร่งใสของเนื้อหาในยุคดิจิทัล
- ช่วยลดการแพร่กระจายของข้อมูลที่ถูกสร้างขึ้นโดยปัญญาประดิษฐ์ในลักษณะที่อาจก่อให้เกิดความเข้าใจผิดหรือการใช้งานที่ผิดวัตถุประสงค์

2. ด้านความเป็นส่วนตัวและความปลอดภัย (Privacy & Security):

- สร้างแนวทางการพัฒนาซอฟต์แวร์ที่ยึดหลักการรักษาความเป็นส่วนตัวของผู้ใช้งานเป็นศูนย์กลาง (Privacy-First) โดยการประมวลผลข้อมูลทั้งหมดภายในเครื่องของผู้ใช้เอง

- ลดความเสี่ยงจากการรั่วไหลของข้อมูลความลับหรือข้อมูลส่วนบุคคลสู่เซิร์ฟเวอร์ภายนอกในระหว่างกระบวนการตรวจสอบ

3. ด้านองค์ความรู้ทางวิศวกรรม (Engineering Knowledge):

- ได้แนวทางการประยุกต์ใช้เทคนิคการบีบอัดแบบจำลอง (Model Quantization) เพื่อเพิ่มประสิทธิภาพการทำงานของโมเดล Deep Learning บนเว็บเบราว์เซอร์
- เป็นกรณีศึกษาในการใช้ ONNX Runtime Web สำหรับการรันโมเดลภาษาขนาดใหญ่บนอุปกรณ์ที่มีทรัพยากรจำกัด (Resource-Constrained Devices)

4. ด้านการพัฒนาอย่างต่อเนื่อง (Sustainability & Improvement):

- ได้ฐานข้อมูลและระบบป้อนกลับ (Feedback Loop) จากผู้ใช้งานจริง เพื่อนำมาปรับปรุงและพัฒนาความแม่นยำของโมเดลให้เท่าทันต่อการเปลี่ยนแปลงของเทคโนโลยีปัญญาประดิษฐ์ในอนาคต

1.5 ข้อจำกัดของโครงการ

ในการดำเนินโครงการนี้ มีข้อจำกัดทางด้านเทคนิคและทรัพยากรที่อาจส่งผลกระทบต่อการทำงานของระบบ ดังนี้:

1. ด้านทรัพยากรของอุปกรณ์ผู้ใช้งาน (Hardware Limitations):

- เนื่องจากระบบประมวลผลบนเว็บเบราว์เซอร์ (Client-side) ประสิทธิภาพและความเร็วในการตรวจจับจึงขึ้นอยู่กับขีดความสามารถของหน่วยประมวลผล (CPU/GPU) และหน่วยความจำ (RAM) ของอุปกรณ์ที่ผู้ใช้งานเป็นหลัก

2. ด้านขนาดของแบบจำลอง (Model Size):

- การนำแบบจำลองภาษาขนาดใหญ่มาใช้งานบนเว็บ จำเป็นต้องผ่านกระบวนการบีบอัด (Quantization) เพื่อให้มีขนาดเหมาะสมต่อการดาวน์โหลด ซึ่งอาจส่งผลให้ความแม่นยำลดลงเล็กน้อยเมื่อเทียบกับแบบจำลองขนาดเต็มที่รันบนเซิร์ฟเวอร์ประสิทธิภาพสูง

3. ด้านขอบเขตของข้อมูล (Data Scope):

- ความแม่นยำในการจำแนกข้อความขึ้นอยู่กับชุดข้อมูล (Dataset) ที่ใช้ในการฝึกฝน (Fine-tuning) ซึ่งครอบคลุมเฉพาะปัญญาประดิษฐ์ค่าย Llama3 ตามที่

ระบุไว้ในขอบเขต หากมีการใช้ปัญญาประดิษฐ์ค่ายอื่นที่มีโครงสร้างภาษาแตกต่างออกไป ประสิทธิภาพในการตรวจจับอาจลดลง แต่แม้ในระยะแรกจะเน้นการทดสอบกับ Llama 3 แต่สถาปัตยกรรมของระบบถูกออกแบบให้รองรับการขยายผล (Scalability) เพื่อฝึกฝนโมเดลกับชุดข้อมูลจากปัญญาประดิษฐ์ค่ายอื่นได้ในอนาคต

4. ด้านลักษณะของข้อความ (Text Characteristics):

- ระบบถูกออกแบบมาเพื่อวิเคราะห์ข้อความที่มีความยาวในระดับหนึ่งเพื่อให้แบบจำลองสามารถดึงคุณลักษณะเชิงภาษา (Stylometry) ออกมาได้ หากข้อความสั้นเกินไปหรือขาดบริบทที่ชัดเจน อาจส่งผลต่อความเชื่อถือได้ของค่าความน่าจะเป็น (Probability) ในการวิเคราะห์

5. ด้านการตัดคำภาษาผสม (Tokenization):

- การจัดการกับข้อความที่มีการสลับภาษาไทยและอังกฤษ (Code-switching) บนหน้าเว็บเบราว์เซอร์ อาจมีความซับซ้อนในขั้นตอนการตัดคำ (Tokenization) ซึ่งขึ้นอยู่กับประสิทธิภาพของ JavaScript Tokenizer ที่พัฒนาขึ้นภายใต้ข้อจำกัดของสภาพแวดล้อมบนเว็บ

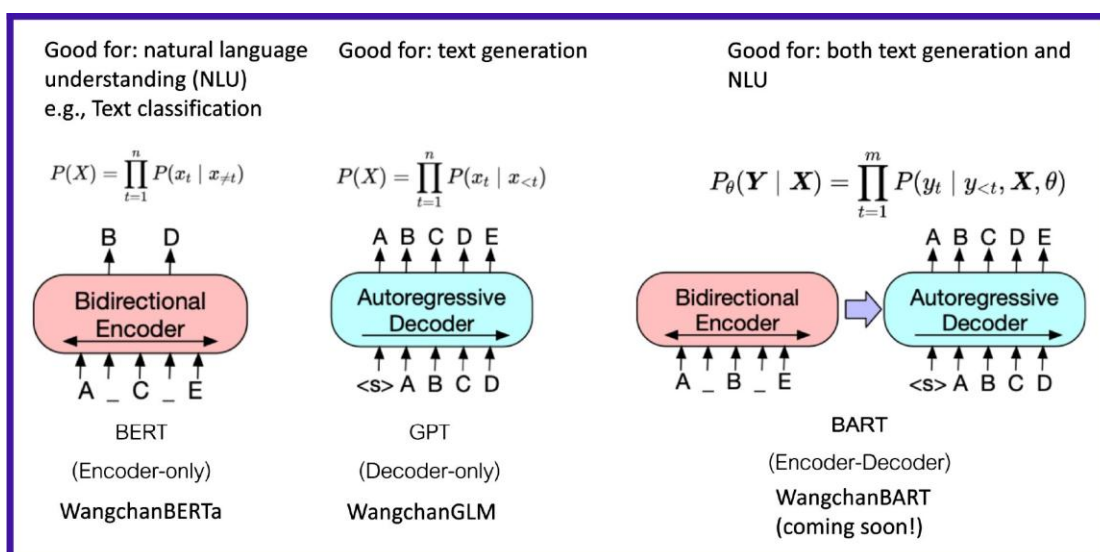
บทที่ 2

วรรณกรรมและงานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดทฤษฎีที่เกี่ยวข้อง

2.1.1 การจัดการภาษาไทยด้วยโมเดล WangchanBERTa

การประมวลผลภาษาไทยมีความซับซ้อนเฉพาะตัว เนื่องจากเป็นภาษาที่เขียนต่อเนื่องกันโดยไม่มีเครื่องหมายแบ่งประโยคหรือช่องว่างระหว่างคำที่ชัดเจน การจะจำแนกข้อความให้แม่นยำจึงต้องใช้โมเดลที่เข้าใจบริบทของภาษาไทยอย่างลึกซึ้ง ผู้วิจัยจึงเลือกจึงเลือกใช้โมเดล WangchanBERTa ซึ่งเป็นโมเดลภาษาขนาดใหญ่ที่ผ่านการฝึกฝนด้วยคลังข้อมูลภาษาไทยที่หลากหลาย สถาปัตยกรรมแบบ RoBERTa ของโมเดลนี้ช่วยให้ระบบสามารถจับใจความความสัมพันธ์ของคำในประโยคได้ดี ซึ่งเป็นหัวใจสำคัญในการแยกแยะความแตกต่างระหว่างสำนวนการเขียนของมนุษย์กับข้อความที่สร้างขึ้นจากโมเดลภาษา



ภาพที่ 2.1 การเปรียบเทียบสถาปัตยกรรมพื้นฐานของโมเดลภาษาขนาดใหญ่ (ที่มา: VISAI, ม.ป.ป.)

การเปรียบเทียบและเหตุผลการเลือกโมเดลในรูปแบบตาราง

ผู้วิจัยเลือกใช้ WangchanBERTa เนื่องจากใช้เทคนิค Byte-level BPE ซึ่งแตกต่างจากโมเดลหลายภาษาตัวอื่น โดย BBPE จะมองข้อมูลเป็นระดับ Byte ทำให้สามารถจัดการกับข้อความภาษาผสม (Thai-English Code-switching) ได้อย่างมีประสิทธิภาพโดยไม่เกิดปัญหา Unknown Token ([UNK]) แม้จะเป็นคำศัพท์เฉพาะทางหรือคำทับศัพท์ที่ไม่ได้อยู่ใน

พจนานุกรมหลัก นอกจากนี้การที่โมเดลถูก Pre-trained ด้วยคลังข้อมูลภาษาไทยขนาดใหญ่ (เช่น Pantip) ทำให้มีความเข้าใจลักษณะการพิมพ์ของมนุษย์ในบริบทสังคมออนไลน์ได้ดีกว่าโมเดลที่เทรนแบบหว่านหลายภาษา

ตารางที่ 2.1 เปรียบเทียบและเหตุผลการเลือกโมเดล

คุณสมบัติ	mBERT / XLM-R	WangchanBERTa (เลือกใช้)
ฐานข้อมูลที่ใช้เทรน	หลายภาษา (Multilingual)	เน้นภาษาไทยเป็นหลัก (Thai-centric)
Tokenizer	WordPiece / SentencePiece	Byte-level BPE (BBPE)
การรองรับ Code-switching	รองรับได้ระดับทั่วไป	ดีเยี่ยม (จัดการ ASCII และอักขระไทยได้ระดับ Byte)
ความเข้าใจบริบทไทย	ปานกลาง (มักมีปัญหาเรื่องการตัดคำผิด)	สูงมาก (เข้าใจสำนวนและโครงสร้างไวยากรณ์ไทย)

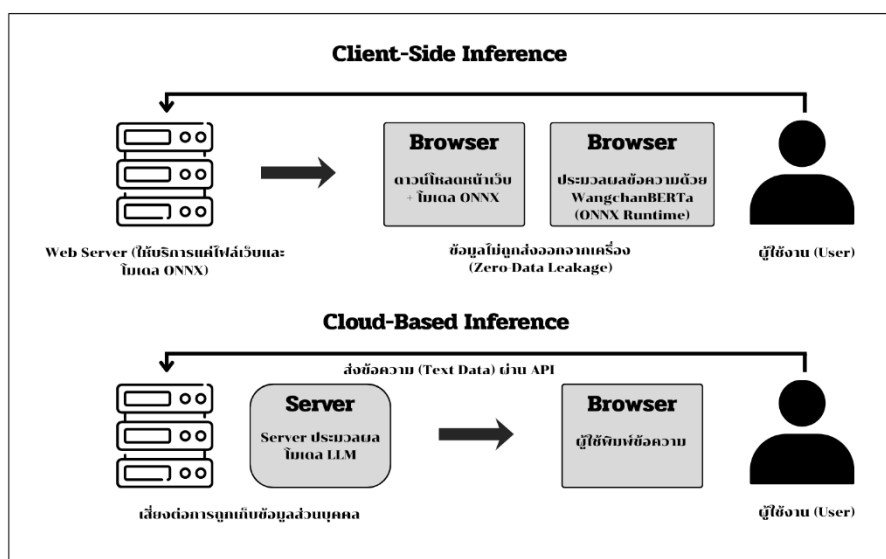
จากตารางที่ 2.1 แสดงให้เห็นว่าแม้แบบจำลองมาตรฐานอย่าง mBERT หรือ XLM-R จะมีความสามารถในการรองรับหลายภาษา แต่ยังมีข้อจำกัดด้านการตัดคำ (Tokenization) เมื่อต้องเผชิญกับโครงสร้างภาษาไทยที่มีความซับซ้อน ผู้วิจัยจึงเลือกใช้ WangchanBERTa เป็นแกนหลักเนื่องจากใช้เทคนิค Byte-level BPE (BBPE) ที่สามารถจัดการอักขระในระดับ Byte ช่วยให้การประมวลผลภาษาไทยผสมอังกฤษ (Code-switching) มีความแม่นยำสูงขึ้นและลดปัญหาการเกิดคำศัพท์นอกพจนานุกรม (Out-of-vocabulary) ได้อย่างมีประสิทธิภาพ

2.1.2 หลักการตรวจจับข้อความที่สร้างโดยปัญญาประดิษฐ์

งานวิจัยด้านนี้มีพื้นฐานมาจากการจำแนกประเภท โดยใช้เทคนิคการเรียนรู้เชิงลึก (Deep Learning) แนวคิดหลักคือการมองหา "ร่องรอย" หรือรูปแบบที่ผิดธรรมชาติ (Artifacts) ที่ปัญญาประดิษฐ์มักจะสร้างขึ้น เช่น การวางโครงสร้างประโยคที่มีความคงที่เกินไป หรือการใช้คำศัพท์ที่ซ้ำซ้อนในเชิงสถิติ โดยโมเดลจะแปลงข้อความเป็นตัวเลข (Vector) และใช้การตัดสินใจแบบ Binary Classification เพื่อประเมินว่าข้อความนั้นมีโอกาสเป็น AI หรือไม่

2.1.3 การประมวลผลฝั่งผู้ใช้งาน (Client-side Intelligence) และความเป็นส่วนตัวของข้อมูล

โดยปกติระบบตรวจจับ AI ส่วนใหญ่มักประมวลผลบนเซิร์ฟเวอร์ (Server-side) ซึ่งผู้ใช้งานต้องส่งข้อความเข้าไปในระบบ ทำให้เกิดความกังวลเรื่องความปลอดภัยของข้อมูลส่วนบุคคล โครงการนี้จึงเปลี่ยนมาใช้สถาปัตยกรรมแบบ Client-side Intelligence โดยย้ายการประมวลผล (Inference) มาไว้บนเว็บเบราว์เซอร์ของผู้ใช้งานโดยตรง วิธีนี้นอกจากจะช่วยลดภาระของเซิร์ฟเวอร์แล้ว ยังเป็นการแก้ปัญหาด้านความเป็นส่วนตัวได้อย่างเด็ดขาด เพราะข้อความทั้งหมดจะถูกประมวลผลและจบอยู่แค่ในอุปกรณ์ของผู้ใช้เท่านั้น (Zero-Data Leakage) ในระหว่างขั้นตอนการประมวลผล (Inference Phase)



ภาพที่ 2.2 แผนภาพเปรียบเทียบสถาปัตยกรรมการอนุมานโมเดลปัญญาประดิษฐ์ระหว่างฝั่งคลาวด์ เซิร์ฟเวอร์และฝั่งผู้ใช้งาน

2.1.4 การบีบอัดแบบจำลอง (Model Quantization) และเทคโนโลยี ONNX Runtime Web

การนำโมเดลภาษาขนาดใหญ่มาใช้งานบนเบราว์เซอร์จริง ๆ มีข้อจำกัดเรื่องทรัพยากรฮาร์ดแวร์ จึงต้องนำเทคนิค Model Quantization มาใช้เพื่อบีบอัดขนาดของค่าน้ำหนัก (Weights) จากรูปแบบทศนิยม (FP32) ให้เป็นจำนวนเต็มเพื่อลดขนาดไฟล์ให้เล็กลง นอกจากนี้ ยังมีการใช้ไลบรารี ONNX Runtime Web เพื่อช่วยให้เบราว์เซอร์สามารถดึงพลัง

การประมวลผลจากเครื่องของผู้ใช้ผ่าน WebAssembly หรือ WebGL มาช่วยรันโมเดลได้โดยตรง ทำให้ระบบโหลดเร็วขึ้นและไม่ใช้หน่วยความจำ (RAM) ของเครื่องมากเกินไป

Weights (32-bit float)	Quantized Weights (8-bit signed int)	Reconstructed Weights (32-bit float)
2.52	121	2.53
-1.12	-54	-1.13
1.74	83	1.73
0.05	2	0.04
0.08	4	0.08
-0.22	-11	-0.23
-1.21	-58	-1.21
2.65	127	2.65
-0.13	-6	-0.13
1.60	77	1.61
0.02	1	0.02
-1.31	-63	-1.32
2.13	102	2.12
-0.01	0	0.00
1.83	88	1.84
1.65	79	1.65

ภาพที่ 2.3 กระบวนการแปลงค่าน้ำหนักของแบบจำลองจากรูปแบบทศนิยม 32 บิต (FP32) เป็นจำนวนเต็ม 8 บิต (INT8) ผ่านเทคนิค Quantization (ที่มา: GeeksforGeeks, ม.ป.ป.)

2.2 งานวิจัยที่เกี่ยวข้อง

2.2.1 การวิเคราะห์ประสิทธิภาพและข้อจำกัดของระบบตรวจจับในปัจจุบัน

ช่วงที่ผ่านมา มีงานวิจัยหลายชิ้นที่ชี้ให้เห็นถึงศักยภาพของสถาปัตยกรรม Transformer ในการแยกแยะแหล่งที่มาของข้อความ เช่น งานของ Mitchell et al. (2023) ที่ระบุว่า การวิเคราะห์ความสอดคล้องของบริบท (Contextual consistency) ให้ผลลัพธ์ที่ดีกว่าวิธีดั้งเดิมอย่างเห็นได้ชัด แต่เมื่อนำมาใช้งานจริงในประเทศไทย พบว่าโมเดลที่ถูกเทรนด้วยข้อมูลจากฝั่งตะวันตกมักจะมีอคติ (Bias) และมีความคลาดเคลื่อนสูงเมื่อเจอกับโครงสร้างประโยคภาษาไทยที่มีความซับซ้อน

นอกจากนี้ งานวิจัยที่ศึกษาประสิทธิภาพของโมเดลภาษาขนาดใหญ่ยังพบข้อสังเกตว่า ถึงแม้ว่าโมเดลจะทำคะแนนความแม่นยำได้ดี แต่ก็ยังขาดความยืดหยุ่นเวลาเจอรูปแบบประโยคที่หนีออกไปจากมาตรฐานที่เคยเรียนรู้มา (Chen & Wan, 2024) อีกทั้งงานของ Winata et al. (2019) ยังยืนยันว่าการสลับภาษา (Code-switching) ก่อให้เกิดความซับซ้อนในการวิเคราะห์ความหมายผสม ซึ่งเป็นจุดที่โมเดลตรวจจับส่วนใหญ่ยังทำได้ไม่ดีพอ

ดังนั้น โครงการนี้จึงมุ่งที่จะแก้ปัญหาเหล่านี้ด้วยการนำแนวคิดของ Lowphansirikul et al. (2021) ในการใช้โมเดล WangchanBERTa ที่ผ่านการฝึกฝนด้วยคลังข้อมูลภาษาไทยขนาดใหญ่และเข้าใจโครงสร้างภาษาอย่างลึกซึ้ง มาทำงานร่วมกับระบบแบบ Client-side เพื่อให้ได้เครื่องมือตรวจจับที่ทั้งแม่นยำและปลอดภัยสำหรับผู้ใช้งานชาวไทย

ตารางที่ 2.2 การวิเคราะห์ช่องว่างทางงานวิจัย

งานวิจัย (Author)	แนวคิดหลัก (Key Concept)	ข้อจำกัด (Limitation)	การเติมเต็มในโครงการนี้ (Contribution)
Mitchell et al. (2023)	การใช้ DetectGPT เพื่อ ตรวจจับข้อความจาก LLM	มุ่งเน้นภาษาอังกฤษ และต้องใช้ทรัพยากร สูงในการคำนวณ Log-likelihood	เน้นภาษาไทยผสม อังกฤษ และประมวลผล แบบ Client-side ที่ใช้ ทรัพยากรน้อย
Lowphansirikul et al. (2021)	การใช้ Byte-level BPE (BBPE) ใน WangchanBERTa	เน้นการประมวลผล ภาษา (NLP) ทั่วไป ไม่ได้ใช้เพื่อการ ตรวจจับ AI	ประยุกต์ใช้ BBPE เพื่อ แก้ปัญหาคำผิดใน ข้อความที่ถูกรสร้างโดย AI
Winata et al. (2019)	การจัดการข้อมูลภาษา ผสม (Code-switching)	เน้นความเข้าใจภาษา (NLU) แต่ไม่มีระบบ แยกแยะแหล่งที่มา	ระบุแหล่งที่มา (Human vs AI) ในบริบทภาษา ผสมได้อย่างแม่นยำ
Saifon et al. (2023)	การวิเคราะห์คุณลักษณะ และพฤติกรรมการใช้ ภาษาไทยบนสื่อสังคม ออนไลน์	โฟกัสเฉพาะข้อความ ที่สร้างโดยมนุษย์ และ ประมวลผลผ่านระบบ เซิร์ฟเวอร์ส่วนกลาง (Server-side)	ขยายขอบเขตการ ตรวจจับไปยังข้อความที่ สร้างโดย AI (Llama 3) และประมวลผลแบบ Client-side เพื่อความ เป็นส่วนตัว

จากการเปรียบเทียบในตารางที่ 2.2 พบว่างานวิจัยส่วนใหญ่ยังขาดการตรวจจับ AI รุ่นใหม่ (Llama 3) บนสภาพแวดล้อมที่รักษาความเป็นส่วนตัว (Client-side) ผู้วิจัยจึงพัฒนาระบบเพื่ออุดช่องว่างดังกล่าว โดยมุ่งเน้นความแม่นยำในการจำแนกภาษาผสมและความเป็นส่วนตัวของข้อมูลบนอุปกรณ์ที่มีทรัพยากรจำกัด

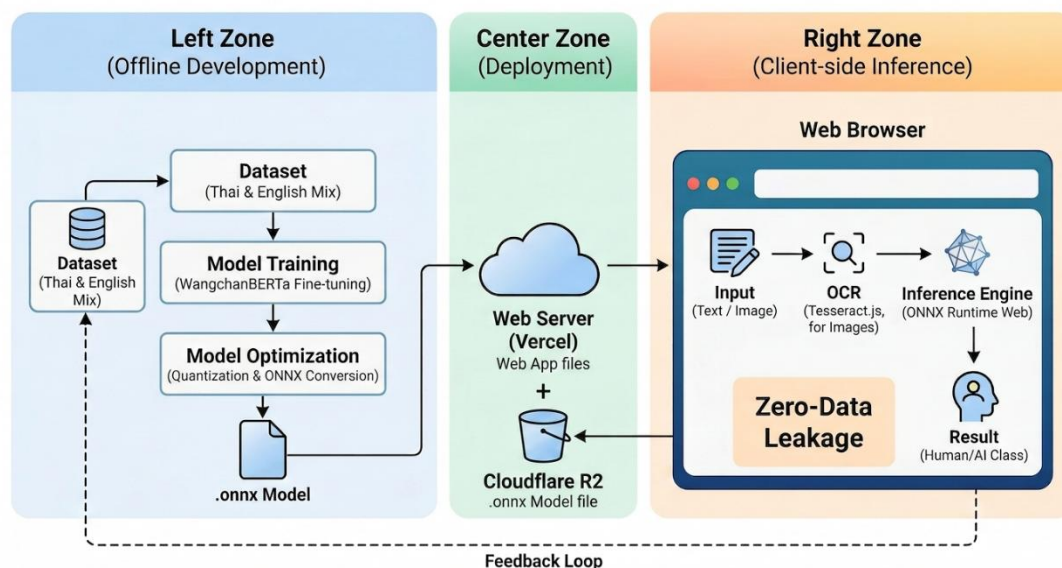
บทที่ 3

วิธีการวิจัย

การพัฒนาระบบตรวจจับข้อความปัญญาประดิษฐ์ในโครงการนี้ มีเป้าหมายสำคัญคือการสร้างสมดุลระหว่างความแม่นยำในการจำแนกภาษาผสม (Thai-English Code-switching) และการรักษาความเป็นส่วนตัวของข้อมูลผู้ใช้งาน ผู้จัดทำจึงได้ออกแบบระเบียบวิธีวิจัยและโครงสร้างระบบที่เน้นการประมวลผลฝั่งผู้ใช้งานเป็นหลัก โดยมีรายละเอียดการดำเนินงานดังนี้

3.1 ภาพรวมของโครงการ

เพื่อให้ระบบบรรลุวัตถุประสงค์ด้านความเป็นส่วนตัวและการประมวลผลที่รวดเร็ว สถาปัตยกรรมของระบบจึงถูกออกแบบให้ทำงานแบบเบ็ดเสร็จบนเบราว์เซอร์ของผู้ใช้งาน (Client-side Intelligence) โดยสามารถแบ่งลำดับขั้นตอนการทำงานตาม ภาพที่ 3.1 ออกเป็น 3 ส่วนหลักดังนี้:



ภาพที่ 3.1 สถาปัตยกรรมภาพรวมของระบบตรวจจับข้อความปัญญาประดิษฐ์

3.1.1 ขั้นตอนการพัฒนาแบบจำลอง (Offline Development Phase)

เริ่มต้นจากการคัดเลือกชุดข้อมูลภาษาไทยและภาษาผสม (Code-switching) เพื่อนำมาทำการปรับแต่ง (Fine-tuning) บนแบบจำลอง WangchanBERTa ผ่านเครื่องคอมพิวเตอร์ประสิทธิภาพสูง เพื่อให้โมเดลจดจำลักษณะเฉพาะของข้อความที่สร้างจากมนุษย์ และ ปัญหาประดิษฐ์ ได้อย่างแม่นยำ

3.1.2 ขั้นตอนการบีบอัดและแปลงรูปแบบ (Model Optimization Phase)

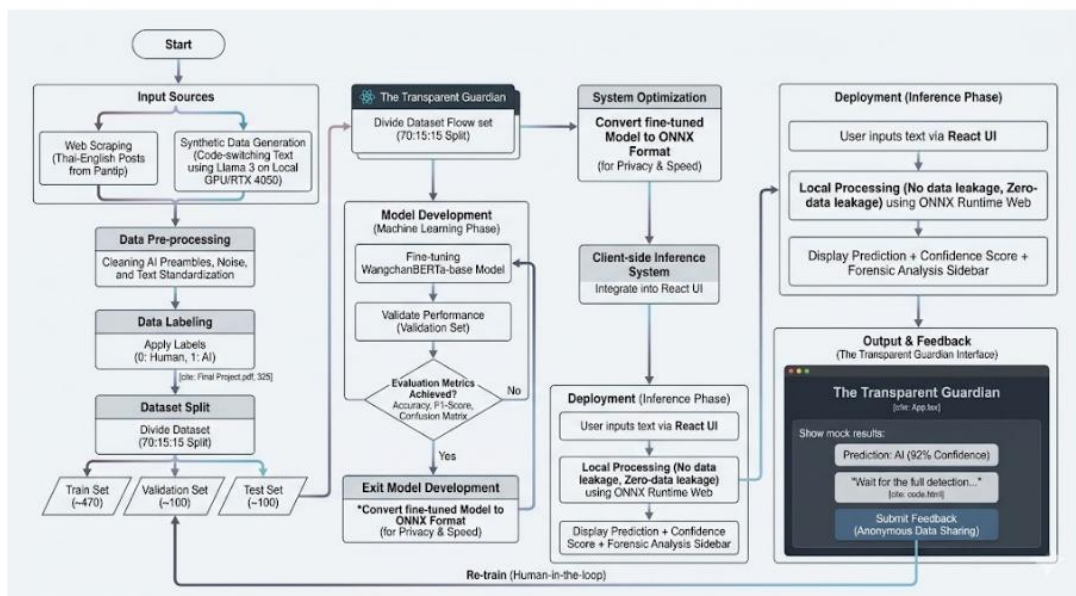
เมื่อได้แบบจำลองที่ผ่านการฝึกฝนแล้ว จะเข้าสู่กระบวนการบีบอัดแบบจำลอง (Quantization) เพื่อเปลี่ยนค่าน้ำหนักจาก FP32 เป็น INT8 ซึ่งช่วยลดขนาดไฟล์ให้เล็กลงโดยยังรักษาความแม่นยำไว้ได้ในระดับสูง จากนั้นจึงแปลงไฟล์ให้อยู่ในรูปแบบ ONNX (.onnx) เพื่อให้พร้อมสำหรับการรันบนสภาพแวดล้อมของเว็บเบราว์เซอร์ จากนั้นไฟล์ .onnx จะถูกอัปโหลดขึ้น Cloudflare R2 Object Storage เพื่อให้บริการผ่าน CDN

3.1.3 ขั้นตอนการอนุมานผลบนเบราว์เซอร์ (Client-side Inference Phase)

ในส่วนของผู้ใช้งาน เมื่อมีการนำเข้าข้อมูลไม่ว่าจะเป็นข้อความหรือรูปภาพ ระบบจะทำการประมวลผลภายในเบราว์เซอร์ทันที โดยหากเป็นรูปภาพจะใช้ไลบรารี Tesseract.js ในการทำ OCR ก่อนจะส่งต่อให้ ONNX Runtime Web ทำการจำแนกแหล่งที่มาของข้อความ กระบวนการนี้จะไม่มีการส่งข้อมูลออกนอกเครื่องของผู้ใช้ (Zero-Data Leakage) เพื่อความปลอดภัยสูงสุดของข้อมูลส่วนบุคคล

3.2 การวิเคราะห์ขอบเขตและความต้องการของระบบ

3.2.1 Flow Chart



ภาพที่ 3.2 แผนภาพกระบวนการเตรียมข้อมูลและการพัฒนาโมเดลตรวจจับข้อความ

กระบวนการดำเนินงานเริ่มต้นจากการรวบรวมข้อมูลจากสองแหล่งหลัก ได้แก่ การขุดข้อมูลจากกระทู้สนทนาบนเว็บไซต์ Pantip และการใช้แบบจำลองภาษาขนาดใหญ่ Llama 3 บนหน่วยประมวลผลกราฟิกส่วนบุคคล (Local GPU) เพื่อสร้างชุดข้อมูลสังเคราะห์ในหัวข้อที่เกี่ยวข้องกัน โดยข้อมูลทั้งหมดจะเข้าสู่ขั้นตอนการประมวลผลเบื้องต้นเพื่อทำความสะอาดและจัดระเบียบข้อความ รวมถึงการลบส่วนเกินที่สร้างโดยปัญญาประดิษฐ์ออกเพื่อให้คงไว้เพียงเนื้อหาหลัก จากนั้นข้อมูลจะถูกนำมาติดป้ายกำกับเพื่อจำแนกประเภทระหว่างมนุษย์และปัญญาประดิษฐ์ ก่อนจะดำเนินการแบ่งชุดข้อมูลออกเป็น 3 ส่วน ได้แก่ ชุดข้อมูลสำหรับฝึกสอน ชุดข้อมูลสำหรับทวนสอบ และชุดข้อมูลสำหรับทดสอบ ตามสัดส่วนร้อยละ 70, 15 และ 15 ตามลำดับ เพื่อเตรียมความพร้อมสำหรับการนำไปฝึกสอนแบบจำลอง WangchanBERTa ในขั้นตอนถัดไป

3.2.2 ความต้องการด้านฟังก์ชันการทำงาน (Functional Requirements)

- ระบบต้องสามารถจำแนกแหล่งที่มาของข้อความได้อย่างน้อย 2 ประเภท คือ ข้อความจากมนุษย์ และข้อความจาก ปัญญาประดิษฐ์
- ระบบต้องรองรับทั้งการรับข้อมูลผ่านการพิมพ์ข้อความโดยตรง และการสกัดข้อความจากรูปภาพ (OCR) เพื่อความสะดวกในการตรวจสอบ

- ระบบต้องมีการแสดงค่าระดับความมั่นใจ (Confidence Score) หรือความน่าจะเป็นในเชิงสถิติเพื่อให้ผู้ใช้สามารถนำไปประกอบการตัดสินใจได้
- ระบบต้องมีกลไกรับข้อมูลป้อนกลับ (Feedback Loop) สำหรับจัดเก็บข้อความที่ระบบจำแนกผิดพลาด เพื่อนำไปพัฒนาความแม่นยำในระยะยาว

3.2.3 ความต้องการด้านคุณภาพและความปลอดภัย (Non-functional Requirements)

- ความเป็นส่วนตัว (Data Privacy): ระบบต้องทำงานภายใต้หลักการ Zero-Data Leakage โดยต้องไม่มีการส่งข้อความของผู้ใช้ไปยังเซิร์ฟเวอร์ภายนอกในระหว่างการประมวลผล
- ประสิทธิภาพ (Performance): ระบบต้องมีความเร็วในการอนุมานผล (Inference) ที่เหมาะสมกับการใช้งานบนเว็บเบราว์เซอร์ โดยไม่ทำให้ทรัพยากรของเครื่องผู้ใช้งานหนักเกินไป
- การเข้าถึงได้ (Accessibility): ส่วนติดต่อผู้ใช้งาน (UI) ต้องมีความเรียบง่าย ไม่ซับซ้อน และสามารถใช้งานผ่านเบราว์เซอร์มาตรฐานได้ทันทีโดยไม่ต้องติดตั้งซอฟต์แวร์เพิ่มเติม

3.2.4 ความต้องการด้านสภาพแวดล้อมทางเทคนิค (Technical Requirements)

- ระบบต้องพัฒนาด้วยภาษา JavaScript หรือ TypeScript เพื่อให้ทำงานร่วมกับเทคโนโลยี ONNX Runtime Web และ Tesseract.js ได้อย่างราบรื่น
- แบบจำลองภาษา (Model) จะต้องถูกบีบอัดให้อยู่ในรูปแบบที่เบราว์เซอร์สามารถดาวน์โหลดมาประมวลผลได้ในระยะเวลาอันสั้น

3.2.5 การรวบรวมและจัดเตรียมชุดข้อมูล (Data Preparation)

ผู้วิจัยได้พัฒนาสคริปต์ Python สำหรับรวบรวมข้อความจาก Pantip.com ผ่านคำสำคัญที่ครอบคลุมหลากหลายหมวดหมู่ เพื่อให้ได้ข้อมูลที่มีลักษณะ Code-switching ที่เป็นธรรมชาติ โดยมีการตรวจสอบข้อมูลซ้ำและคัดกรองข้อความที่มีความยาวมากกว่า 200 ตัวอักษร เพื่อคุณภาพในการเรียนรู้ของโมเดล เกณฑ์ดังกล่าวส่งผลให้ข้อความที่ผ่านการคัด

กรองมีความยาวเฉลี่ย 969–1,893 ตัวอักษร (ตารางที่ 3.1) ซึ่งเพียงพอต่อการดึงคุณลักษณะ Stylometry ของแบบจำลอง

ตารางที่ 3.1 สรุปรายละเอียดสถิติของชุดข้อมูล

รายละเอียด	Human (Pantip)	AI (Llama 3)	รวม
จำนวนตัวอย่าง (Samples)	335	335	670
ความยาวเฉลี่ย (Avg. Characters)	1,893	969	-
สัดส่วนภาษาผสม (CS Ratio %)	5.31%	4.99%	-
การแบ่งข้อมูล (Train : Val : Test)	-	-	469 / 100 / 101

หมายเหตุ: ข้อมูลทั้งหมด 670 ชุด ถูกแบ่งออกเป็นชุดฝึกสอน (Training Set) 469 ชุด (70%), ชุดทดสอบ (Validation Set) 100 ชุด (15%) และชุดทดสอบ (Test Set) 101 ชุด (15%)

จากการวิเคราะห์สถิติพบว่าข้อความจากมนุษย์มีความยาวเฉลี่ยมากกว่า AI เกือบ 2 เท่า (1,893 ต่อ 969 ตัวอักษร) ในขณะที่ระดับความเข้มข้นของภาษาผสม (CS Ratio) มีความใกล้เคียงกันที่ประมาณ 5% ซึ่งเป็นระดับที่ท้าทายในการจำแนก

ในการจัดเตรียมชุดข้อมูล ผู้วิจัยได้ดำเนินการทำความสะอาดข้อมูลเพื่อลดอคติ (Bias) จากรูปแบบการสร้างข้อความของปัญญาประดิษฐ์ ดังตัวอย่างในตารางที่ 3.2

ตารางที่ 3.2 ตัวอย่างการประมวลผลข้อมูลเพื่อขจัดสิ่งเจือปน (Data Cleaning Examples)

ตัวอย่างที่	ข้อความเริ่มต้น (Raw Text)	ข้อความหลังการประมวลผล (Cleaned Text)	สิ่งที่ถูกลบออก (Noise/Artifacts)
1	"Hey fans! มาสนใจโปรเจกต์พิเศษ ดีดีจาก BNK48 & CGM48 กัน! ... หลังจากสิ้นสุดการโหวต BNK Token ของคุณจะถูก Burn ที่ Don't miss out this opportunity to show your love and support for these talented girls!"	"มาสนใจโปรเจกต์ พิเศษจาก BNK48 & CGM48 กัน! ... หลังจากสิ้นสุดการ โหวต BNK Token ของคุณจะถูก Burn ทิ้ง"	1. คำเกริ่นนำ (Greeting): "Hey fans!" 2. คำสรวล AI: "ดีดี" 3. ประโยคปิดท้าย ภาษาอังกฤษ: "Don't miss out..."
2	"สวัสดี gang! วันนี้ผมมาแชร์ ประสบการณ์ตั้ง Azure Cosmos DB กันครับ ... [connection refused calling hello ... สงสัยมัน เพราะ Network Settings ไม่ ถูกต้องใช่ไหมคะ?"]	"วันนี้ผมมาแชร์ ประสบการณ์ตั้ง Azure Cosmos DB กันครับ ... [connection refused calling hello ... สงสัยมัน เพราะ Network Settings ไม่ถูกต้อง ใช่ไหมคะ?"]	คำเกริ่นนำที่แสดง ลักษณะของ AI Chatbot: "สวัสดี gang!"
3	"มาแล้วจ้า MV เพลงรอง ... เป็น MV ที่ดูแล้วอยากดูซ้ำทันที เอาไป 10/10 This MV got me hook! The energy is high and the vibe is so cool! Can't help but wanna rewatch it over and over again."	"มาแล้วจ้า MV เพลงรอง ... เป็น MV ที่ดูแล้วอยากดู ซ้ำทันที เอาไป 10/10"	ส่วนสรุปเนื้อหาเป็น ภาษาอังกฤษ (Translation/Summary) ที่ AI มักสร้างต่อท้าย ข้อความภาษาไทย

จากการทำความสะอาดข้อมูล พบว่าปัญญาประดิษฐ์ (Llama 3) มักแทรกคำเกริ่นนำ และประโยคปิด เป็นภาษาอังกฤษที่อยู่นอกเหนือจากเนื้อหาหลัก ผู้วิจัยจึงใช้สคริปต์ Python ในการขจัดสิ่งเจือปนเหล่านี้ออก เพื่อให้แบบจำลอง WangchanBERTa สามารถเรียนรู้เฉพาะโครงสร้างภาษาผสม (Code-switching) ที่เป็นเนื้อหาสำคัญจริง ๆ ซึ่งจะช่วยเพิ่มความแม่นยำในการจำแนกแหล่งที่มาของข้อความ

3.3 ประเด็นที่น่าสนใจและสิ่งที่ท้าทาย

3.3.1 การรักษาสมดุลระหว่างขนาดและความแม่นยำ (Size vs. Accuracy Trade-off)

เนื่องจากโครงการนี้เลือกใช้แบบจำลอง WangchanBERTa ซึ่งมีพารามิเตอร์จำนวนมาก การจะทำให้เบราร์เซอร์รันโมเดลนี้ได้จริงจึงต้องใช้เทคนิค Quantization เพื่อลดขนาดไฟล์ อย่างไรก็ตาม การบีบอัดข้อมูลน้ำหนักโมเดลอาจส่งผลให้ความแม่นยำลดลงเล็กน้อย ผู้จัดทำจึงต้องทำการทดสอบเปรียบเทียบค่าความผิดพลาดเพื่อให้แน่ใจว่าผลลัพธ์ที่ได้ยังอยู่ในเกณฑ์ที่ยอมรับได้

3.3.2 ความซับซ้อนของภาษาผสม (Tokenization Challenges)

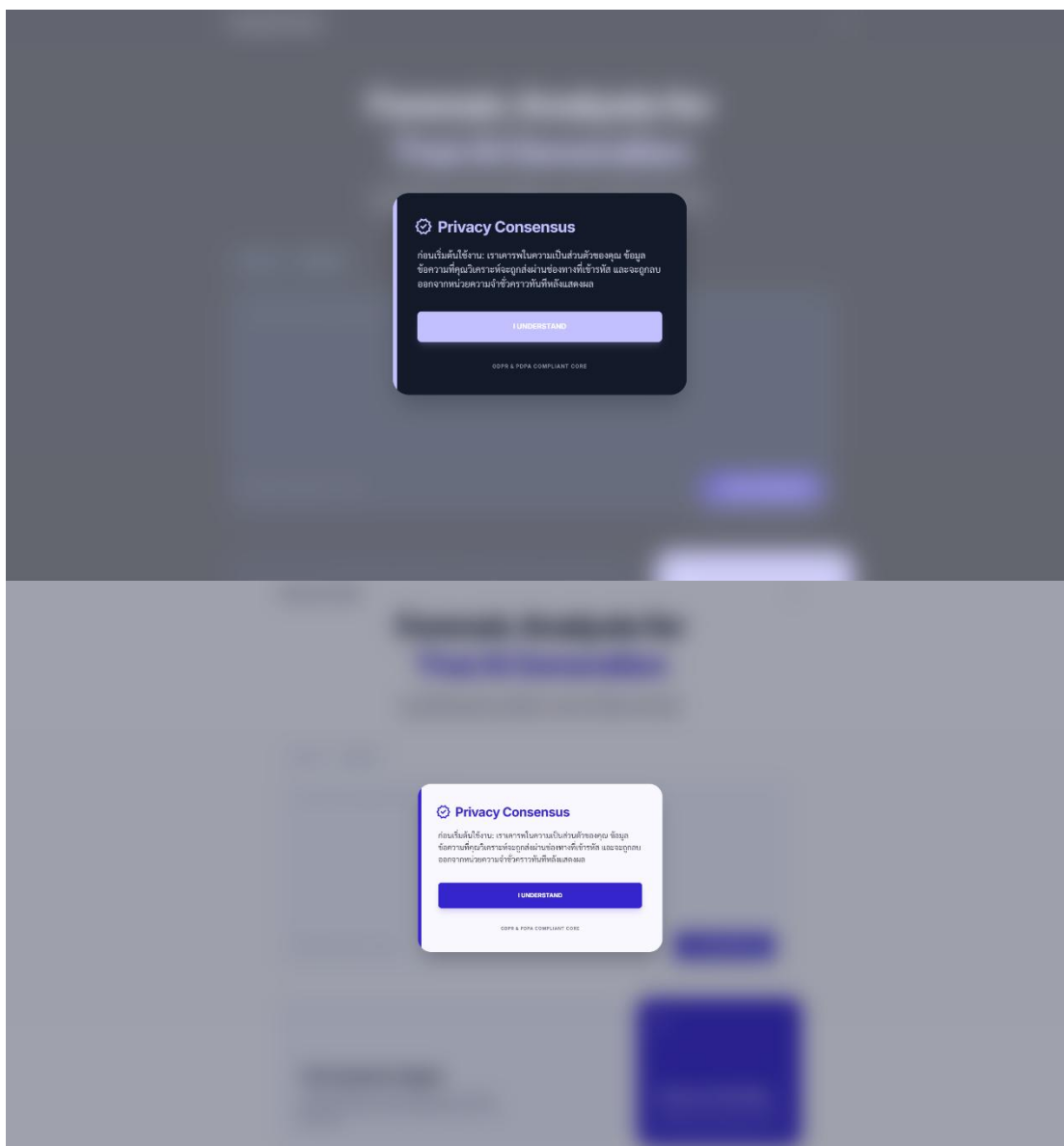
การตัดคำ (Tokenization) ในประโยคที่มีการสลับภาษาไปมาเป็นเรื่องที่ซับซ้อน หากตัวตัดคำบนเบราร์เซอร์ทำได้ไม่ดีพอ จะทำให้ความหมายเชิงอรรถศาสตร์ (Semantics) ผิดเพี้ยนไป โครงการนี้จึงต้องให้ความสำคัญกับการจัดการชุดข้อมูลฝึกฝนให้ครอบคลุมบริบทการสื่อสารในรูปแบบนี้โดยเฉพาะ

3.4 ผลลัพธ์ที่คาดหวัง

ผู้จัดทำคาดหวังว่าระบบต้นแบบ จะเป็นเครื่องมือที่มีประสิทธิภาพในการระบุแหล่งที่มาของข้อความภาษาผสม โดยสร้างความเชื่อมั่นให้กับผู้ใช้งานในด้านความปลอดภัยของข้อมูลส่วนตัว และเป็นแนวทางในการนำโมเดลภาษาขนาดใหญ่มาประยุกต์ใช้งานบนอุปกรณ์ปลายทางในอนาคต

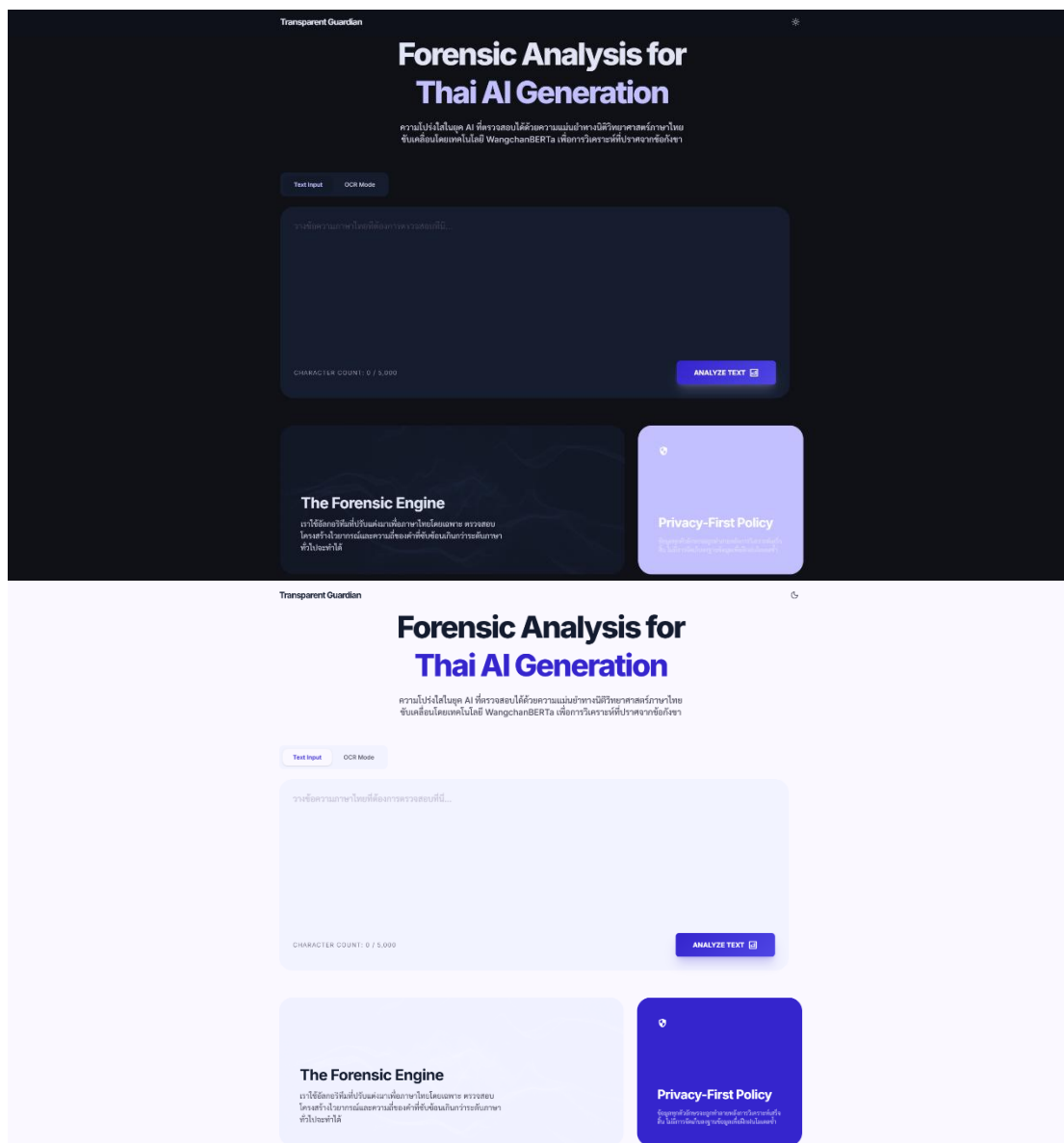
3.5 ระบบต้นแบบและผลลัพธ์เบื้องต้น

3.5.1 ตัวอย่างภาพรวมของระบบ



ภาพที่ 3.3 แสดงหน้าต่างขอความยินยอมในการจัดการข้อมูล (Privacy Consensus) ในโหมดสว่างและโหมดมืด

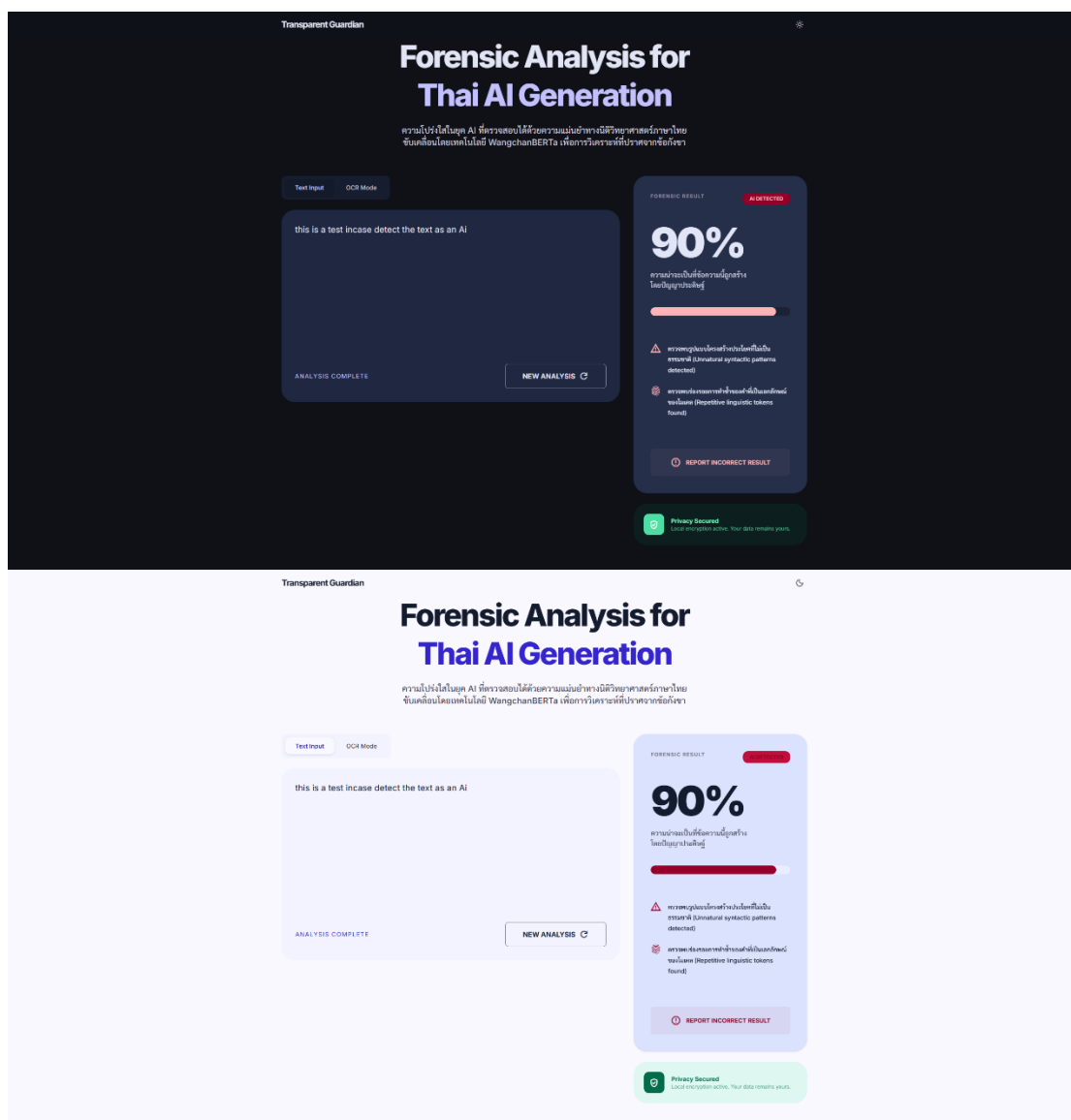
แสดงหน้าต่างแจ้งนโยบายความเป็นส่วนตัวเป็นส่วนตัวเพื่อให้ผู้ใช้รับทราบถึงขอบเขตการทำงานของระบบ โดยการยืนยันตัวตนนี้เป็นส่วนหนึ่งของสถาปัตยกรรมแบบ Privacy-by-Design ที่เน้นย้ำถึงการจัดการข้อมูลตามหลักมาตรฐานสากลก่อนเริ่มกระบวนการประมวลผล



ภาพที่ 3.4 แสดงหน้าหลักของระบบ (Homepage) ในโหมดสว่างและโหมดมืด

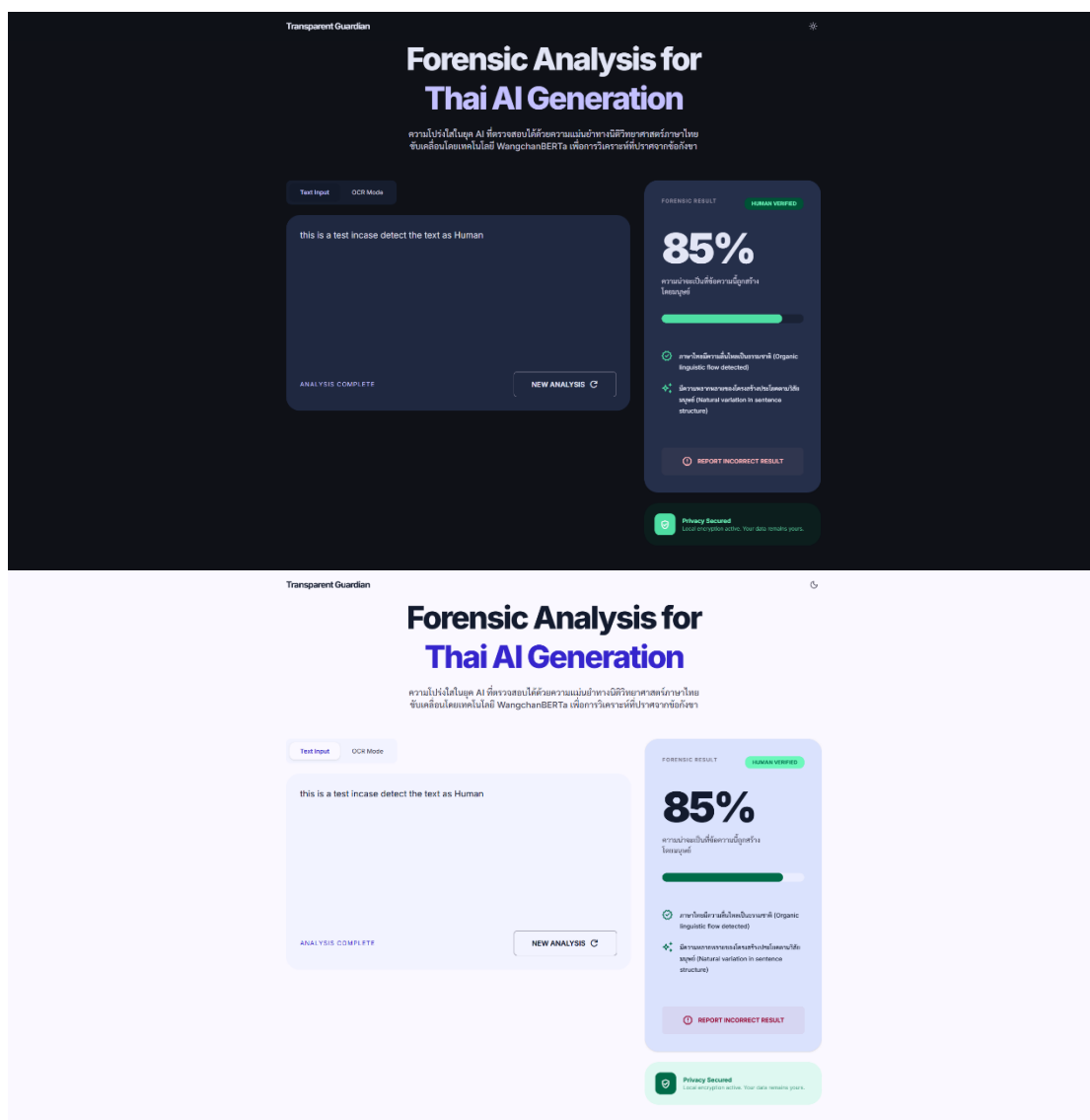
แสดงส่วนติดต่อผู้ใช้งานหลักที่รองรับทั้งโหมดสว่างและโหมดมืด เพื่อส่งเสริมประสบการณ์ผู้ใช้งาน (UX) และลดอุปสรรคในการเข้าถึงข้อมูลผ่านหน้าเว็บแอปพลิเคชันที่ประมวลผลแบบเบราร์โเวอร์เบส

3.5.2 หน้าแสดงผลตัวอย่างการจำแนกข้อความ



ภาพที่ 3.5 แสดงผลการทดสอบข้อความที่สร้างโดยปัญญาประดิษฐ์ (AI Text) ในโหมดสว่างและโหมดมืด

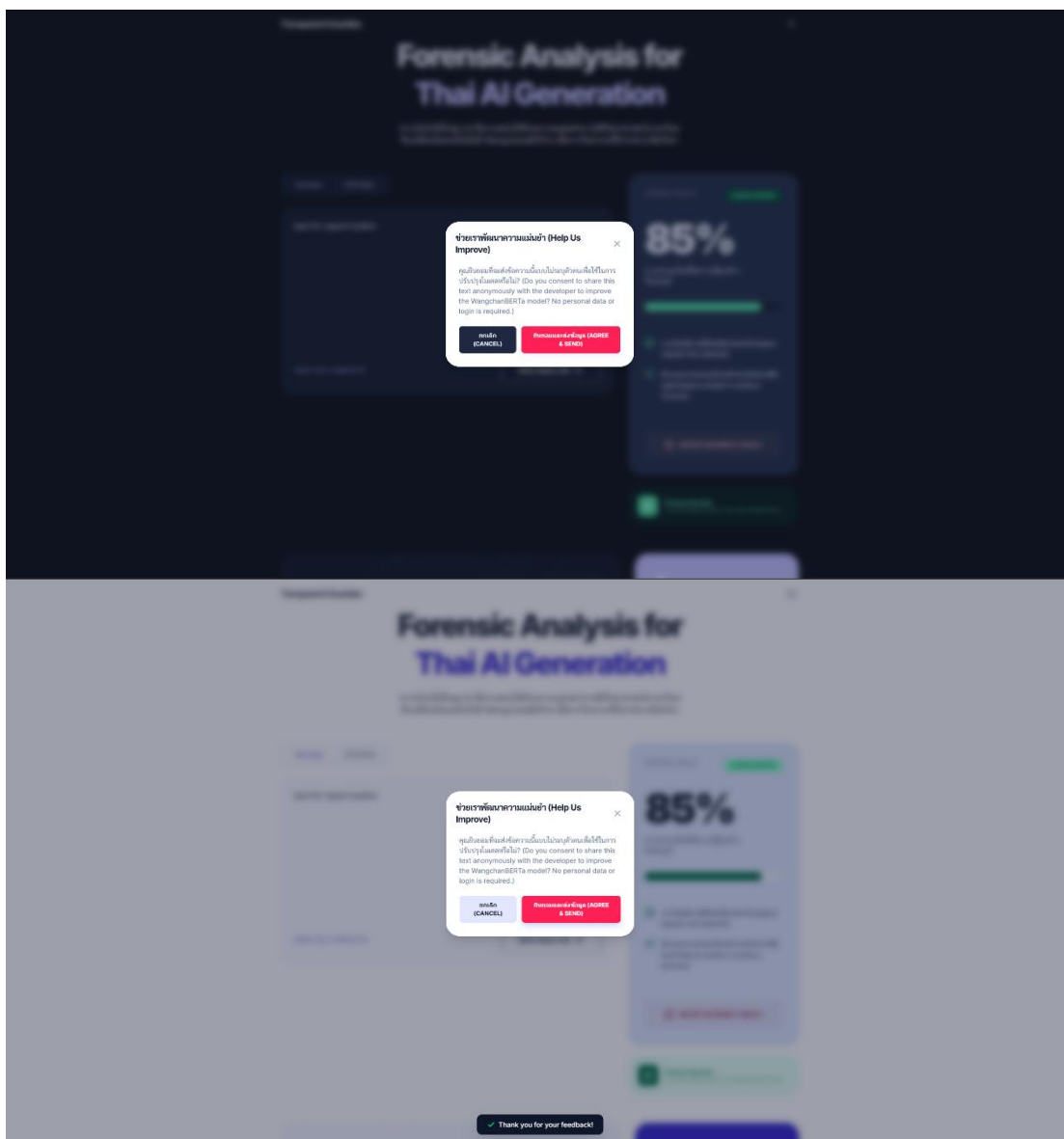
แสดงผลการวิเคราะห์แหล่งที่มาพร้อมค่า Confidence Score ซึ่งเป็นผลลัพธ์ที่ได้จากการทำ Inference ผ่านแบบจำลอง WangchanBERTa บน ONNX Runtime Web ภายในอุปกรณ์ของผู้ใช้งานโดยตรงโดยไม่ผ่านเซิร์ฟเวอร์ภายนอก



ภาพที่ 3.6 แสดงผลการทดสอบข้อความที่เขียนโดยมนุษย์ (Human Text) ในโหมดสว่างและโหมดมืด

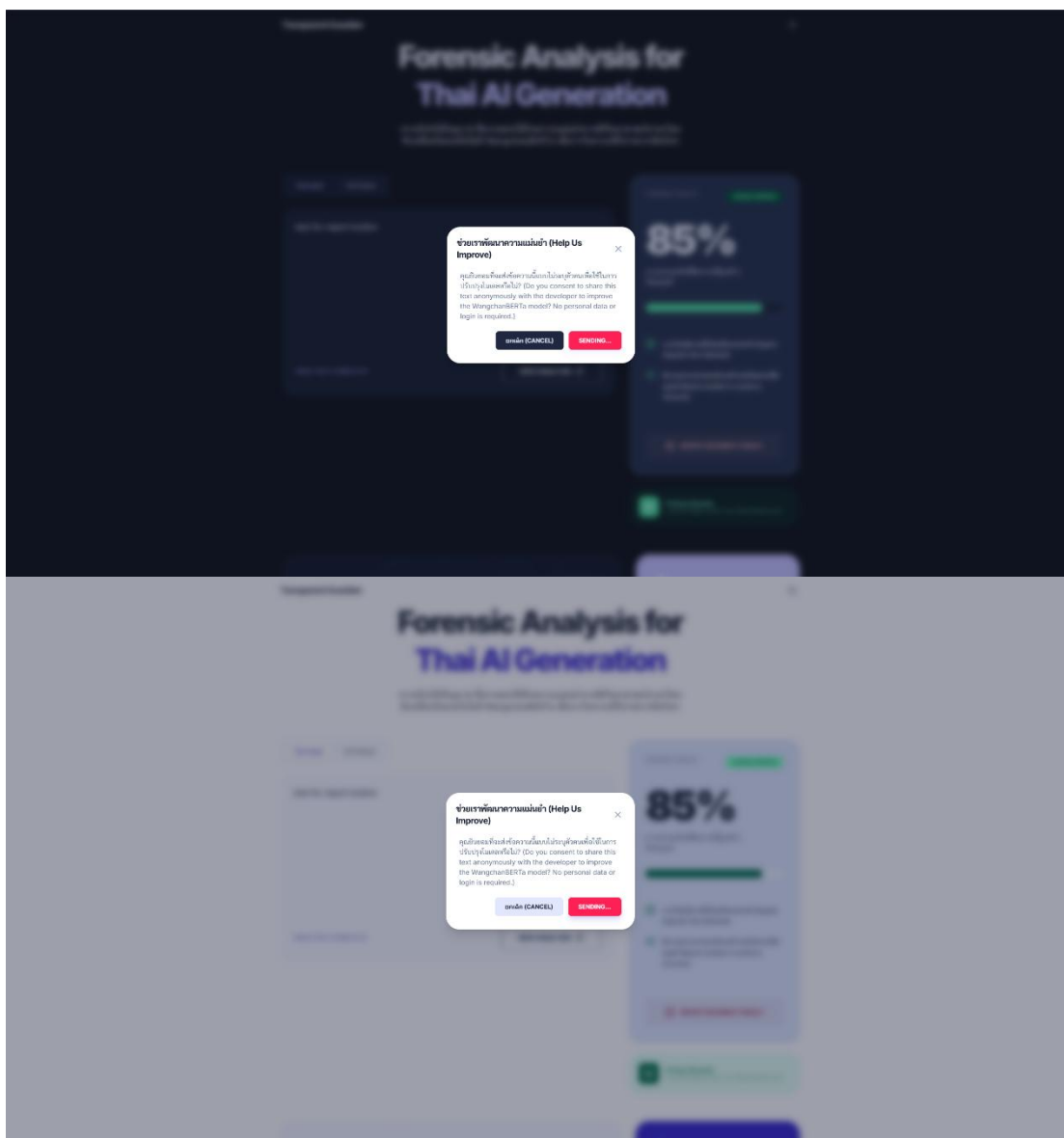
แสดงผลการจำแนกข้อความจากมนุษย์ที่ผ่านกระบวนการ Pre-processing เพื่อลดสัญญาณรบกวน (Noise) ช่วยให้โมเดลสามารถดึงคุณลักษณะเด่นของภาษาผสมมาวิเคราะห์ได้อย่างแม่นยำตามเกณฑ์ที่กำหนดไว้

3.5.3 ระบบรับข้อมูลป้อนกลับและการรายงานผล



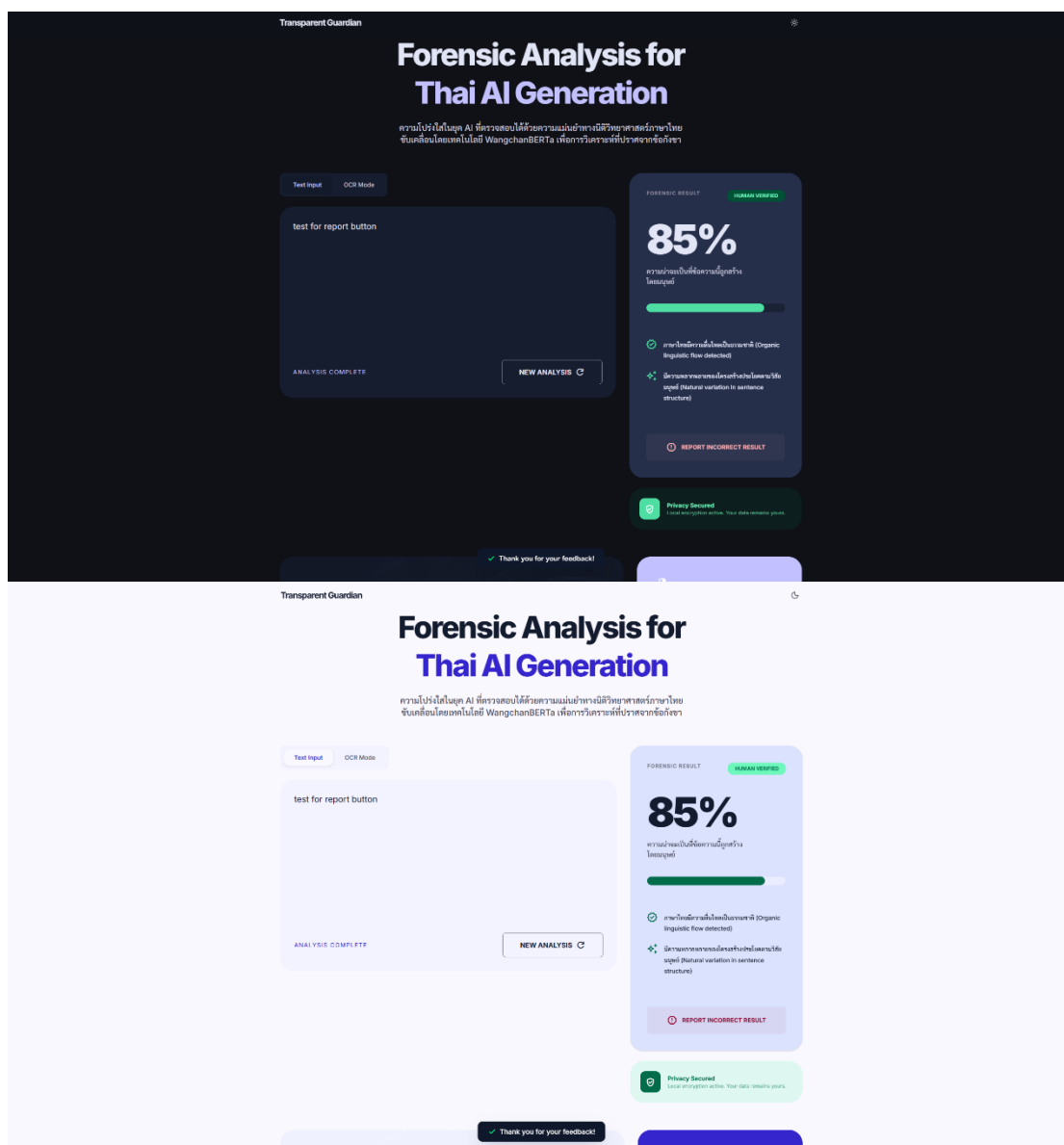
ภาพที่ 3.7 แสดงหน้าเว็บการกดปุ่มรายงานผล (Press report button) ในโหมดสว่างและโหมดมืด

แสดงส่วนการเข้าถึงฟังก์ชันรายงานผล เพื่อสร้าง Active Learning Loop ในการรับข้อมูลจากผู้ใช้งานจริงมาช่วยตรวจสอบความถูกต้องของการทำนายผลในสภาพแวดล้อมที่หลากหลาย



ภาพที่ 3.8 แสดงหน้าเว็บหลังจากกดปุ่มรายงานผล (After press report button) ในโหมดสว่างและโหมดมืด

แสดงหน้าต่างรับข้อมูลป้อนกลับเชิงลึก เพื่อใช้เป็นชุดข้อมูลในการทำ Fine-tuning และปรับปรุงประสิทธิภาพของแบบจำลองให้สามารถจัดการกับรูปแบบประโยคที่ซับซ้อนในอนาคตได้ดียิ่งขึ้น



ภาพที่ 3.9 แสดงการแจ้งเตือนเมื่อดำเนินการส่งข้อมูลสำเร็จ (Thank you message) ในโหมดสว่างและ โหมดมืด

แสดงสถานะการบันทึกข้อมูลป้อนกลับสำเร็จ เพื่อยืนยันความสำเร็จของกระบวนการส่งข้อมูลกลับมายังผู้พัฒนาสำหรับการตรวจสอบและปรับปรุงระบบในลำดับถัดไป

บทที่ 4

ทรัพยากรและแผนการดำเนินงาน

4.1 การจัดเตรียมฮาร์ดแวร์และซอฟต์แวร์

4.1.1 ฮาร์ดแวร์ (Hardware)

- Acer Predator Helios Neo 16 : หน่วยประมวลผล Intel Core i7-13700HX, หน่วยความจำ (RAM) 32GB, และหน่วยประมวลผลกราฟิก NVIDIA GeForce RTX 4050 สำหรับใช้ในการจัดเตรียมชุดข้อมูล ทดสอบการประมวลผลแบบจำลองภาษา และการพัฒนาส่วนติดต่อผู้ใช้งาน

4.1.2 ซอฟต์แวร์ (software and tool)

- Runtime: Node.js เวอร์ชัน 22.x LTS สำหรับเป็นสภาพแวดล้อมในการรันแอปพลิเคชันฝั่งเครื่องผู้ใช้งาน
- Frontend Framework: React ร่วมกับ Vite เพื่อความรวดเร็วในการแสดงผลและการจัดการ State ของแอปพลิเคชัน
- Styling: Tailwind CSS v4 สำหรับการออกแบบส่วนติดต่อผู้ใช้งานที่รองรับทั้งโหมดสว่างและโหมดมืด (Light/Dark Mode)
- Machine Learning Runtime: ONNX Runtime Web สำหรับการนำแบบจำลองภาษามาประมวลผลบนเบราว์เซอร์ (Client-side Inference)
- Programming Language: Python 3.11 สำหรับใช้ในกระบวนการฝึกฝนแบบจำลอง (Fine-tuning) และการทำความสะอาดชุดข้อมูล (Data Pre-processing)
- Database & API: Supabase (PostgreSQL) สำหรับรองรับระบบฐานข้อมูลและการจัดการ API ในส่วนของระบบแจ้งผลตอบรับ (User Feedback Loop)
- Version Control: GitHub สำหรับการบริหารจัดการเวอร์ชันของซอร์สโค้ดตลอดระยะเวลาการดำเนินงาน

- **Model Storage:** Cloudflare R2 สำหรับจัดเก็บและให้บริการไฟล์แบบจำลอง (.onnx) ผ่าน CDN เพื่อเพิ่มความเร็วในการดาวน์โหลดโมเดลสำหรับผู้ใช้งานในประเทศไทย

4.2 แผนการดำเนินงาน

ตารางที่ 4.1 การดำเนินงานที่ผ่านมาจนถึงปัจจุบัน

[illegible]

บทที่ 5

สรุป

5.1 สรุปผลการดำเนินงาน

จากการดำเนินงานในระยะแรกของโครงการระบบตรวจจับข้อความที่สร้างโดยปัญญาประดิษฐ์ภาษาไทย ผู้พัฒนาได้วางรากฐานทางเทคนิคและออกแบบส่วนติดต่อผู้ใช้ (UI) จนได้ระบบต้นแบบที่พร้อมสำหรับการพัฒนาต่อยอด โดยสิ่งที่ทำสำเร็จแล้วประกอบด้วยการสร้างเว็บแอปพลิเคชันด้วย React และ Tailwind CSS v4 ที่มีหน้าตาเรียบง่าย ทันสมัย และรองรับการแสดงผลบนอุปกรณ์ต่างๆ ได้อย่างถูกต้อง ระบบต้นแบบนี้สามารถแสดงผลการวิเคราะห์ระดับความมั่นใจ (Confidence Score) ในรูปแบบเปอร์เซ็นต์ได้อย่างแม่นยำตามการออกแบบ

นอกจากส่วนของการแสดงผล ผู้พัฒนายังได้ให้ความสำคัญกับการออกแบบกลไกข้อมูลป้อนกลับ (Feedback Loop) ร่วมกับการแสดงหน้าต่างขอความยินยอม (Consent Modal) เพื่อให้มั่นใจว่าการจัดเก็บข้อมูลเพื่อนำมาพัฒนาโมเดลในอนาคตจะเป็นไปตามหลักจริยธรรมข้อมูลและความเป็นส่วนตัวของผู้ใช้งาน ปัจจุบันซอร์สโค้ดและโครงสร้างโปรเจกต์ทั้งหมดถูกบริหารจัดการผ่าน GitHub อย่างเป็นระบบ ซึ่งถือเป็นความคืบหน้าที่สำคัญก่อนจะเข้าสู่ขั้นตอนการเชื่อมต่อโมเดล WangchanBERTa ในระยะถัดไป

5.2 อภิปรายผลและข้อจำกัด

จากการทดสอบระบบต้นแบบในเบื้องต้น พบประเด็นที่ท้าทายในด้านการจัดการทรัพยากรการประมวลผล เนื่องจากการนำโมเดลภาษาขนาดใหญ่ (LLM) มาทำ Inference บนเบราว์เซอร์โดยตรง (Client-side) อาจส่งผลกระทบต่อประสิทธิภาพการทำงานในอุปกรณ์ที่มีสมรรถนะจำกัด นอกจากนี้ ในด้านของเนื้อหาภาษาไทยยังพบว่า ข้อความที่ถูกสร้างขึ้นโดย AI มีความแนบเนียนและซับซ้อนขึ้นอย่างต่อเนื่อง ทำให้การจำแนกประเภทข้อความในบางบริบททำได้ยากขึ้น ความท้าทายเหล่านี้จึงเป็นปัจจัยสำคัญที่ผู้พัฒนาจะต้องนำไปพิจารณาในการปรับแต่งโมเดล (Optimization) และการคัดเลือกชุดข้อมูลที่หลากหลายมากขึ้นเพื่อรักษามาตรฐานความแม่นยำของระบบ

5.3 ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต

สำหรับแผนการดำเนินงานในระยะถัดไป ผู้พัฒนาจะมุ่งเน้นไปที่การบูรณาการระบบ OCR โดยใช้ Tesseract.js เพื่อให้ผู้ใช้งานสามารถอัปโหลดข้อมูลผ่านไฟล์รูปภาพได้โดยตรง ซึ่งจะช่วยเพิ่มขีดความสามารถและความสะดวกในการใช้งาน ที่สำคัญที่สุดคือการเชื่อมต่อโมเดล WangchanBERTa ผ่านรูปแบบ ONNX เพื่อเพิ่มความเร็วในการประมวลผลฝั่งผู้ใช้งาน สุดท้ายนี้ ผู้พัฒนาตั้งใจจะนำข้อมูลจริงที่ได้รับจากส่วน Feedback ของผู้ใช้ มาทำการปรับแต่งโมเดล (Fine-tuning) อย่างสม่ำเสมอ เพื่อให้ระบบมีความเท่าทันต่อเทคโนโลยี Generative AI ที่พัฒนาไปอย่างรวดเร็ว และสามารถเป็นเครื่องมือตรวจสอบความน่าเชื่อถือของเนื้อหาดิจิทัลได้อย่างมีประสิทธิภาพ

รายการอ้างอิง

- [1] Lowphansirikul, L., Phuangpham, P., Tarsuilev, P., & Chuntee, S. (2021). WangchanBERTa: Pretrained-language model for Thai. *arXiv preprint arXiv:2101.09635*.
- [2] Mitchell, E., Yoon, C., Liang, P., & Manning, C. D. (2023). DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. *arXiv preprint arXiv:2301.11305*.
- [3] Winata, G. I., et al. (2019). ERC: Enriched Resource for Code-switching Thai-English. *Proceedings of the 5th Workshop on Noisy User-generated Text (W-NUT)*.
- [4] Chen, Y., & Wan, X. (2024). LLM-Detector: Improving AI-Generated Text Detection with Large Language Models. *arXiv preprint arXiv:2402.01234*.
- [5] Saifon, S., & Rakwithoon, N. (2023). Linguistic Features of Thai Social Media: A Study of Pantip and Twitter. *Journal of Thai Language and Culture*, 10(2), 45-60.
- [6] Meta AI. (2024). Llama 3: Open Foundation and Fine-Tuned Chat Models. [Online]. Available: <https://ai.meta.com/llama/>
- [7] Microsoft. ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator. [Online]. Available: <https://onnxruntime.ai/>
- [8] Facebook Open Source. React: A JavaScript library for building user interfaces. [Online]. Available: <https://react.dev/>
- [9] Tailwind Labs. Tailwind CSS: A utility-first CSS framework for rapidly building custom designs. [Online]. Available: <https://tailwindcss.com/>
- [10] Vercel. Vercel: Develop. Preview. Ship. [Online]. Available: <https://vercel.com/>

ภาคผนวก

ภาคผนวก ก

สคริปต์การรวบรวมและทำความสะอาดข้อมูล

ก.1 สคริปต์รวบรวมข้อมูลจาก Pantip.com

สคริปต์ค้นหาโพสต์จาก Pantip.com ด้วย keyword รายการ 53 คำ (บรรทัด 10) วงเล็บที่ใส่ keyword/page ด้วย DELAY_SECONDS = 1.5 เพื่อหลีกเลี่ยง rate limit กรองโพสต์ที่ซ้ำด้วย topic_id และข้ามโพสต์สั้นกว่า 200 ตัวอักษร บันทึกเป็น CSV ทุก 20 รายการ

```
import requests
from bs4 import BeautifulSoup
import csv
import time
import os
import re
import sys

# Configuration
KEYWORDS = ['AI', 'Work from home', 'Hybrid working',
'Meeting', 'Deadline', 'Feedback', 'Soft skills',
'Hard skills', 'Upskill', 'Reskill', 'Growth
mindset', 'Experience', 'Solution', 'Connection',
'Networking', 'Startup', 'Innovation',
'Leadership', 'Teamwork', 'Communication', 'Productivity',
'Work-life balance', 'Remote work', 'Career
development', 'Job search', 'Interview tips', 'Salary negotiation',
'Personal branding', 'Time management', 'Stress
management', 'Mental health at work', 'Diversity and inclusion',
'Employee engagement', 'Company culture',
'Performance review', 'Professional development', 'Mentorship',
'Workplace conflict', 'Collaboration',
'Creativity', 'Problem-solving', 'Decision-making', 'Emotional
intelligence',
'Adaptability', 'Resilience', 'Critical
thinking', 'Innovation in the workplace', 'Future of work', 'Gig
economy',
'Freelancing', 'Entrepreneurship', 'Workplace
trends', 'Digital transformation', 'Automation and AI in the
workplace']

SEARCH_URL = "https://pantip.com/search"
BASE_URL = "https://pantip.com"
OUTPUT_FILE = "pantip_dataset.csv"
TOTAL_POST_LIMIT = 1000
PAGES_PER_KEYWORD = 50
DELAY_SECONDS = 1.5
SAVE_BATCH_SIZE = 20

HEADERS = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64;
x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124
Safari/537.36',
```

```

    }

    def clean_text(text):
        if not text: return ""
        text = re.sub(r'\s+', ' ', text)
        return text.strip()

    def extract_topic_id(url):
        match = re.search(r'/topic/(\d+)', url)
        return match.group(1) if match else None

    def load_seen_ids():
        seen_ids = set()
        if os.path.exists(OUTPUT_FILE):
            try:
                with open(OUTPUT_FILE, 'r', encoding='utf-8-
sig') as f:
                    reader = csv.DictReader(f)
                    for row in reader:
                        if 'topic_id' in row:
                            seen_ids.add(row['topic_id'])
                    print(f"Loaded {len(seen_ids)} existing topic
IDs from {OUTPUT_FILE}")
            except Exception as e:
                print(f"Warning: Could not read existing IDs:
{e}")

        return seen_ids

    def save_to_csv(data_list, mode='a'):
        if not data_list: return
        file_exists = os.path.exists(OUTPUT_FILE)
        keys = data_list[0].keys()
        try:
            with open(OUTPUT_FILE, mode, newline='',
encoding='utf-8-sig') as f:
                dict_writer = csv.DictWriter(f,
fieldnames=keys)
                if not file_exists or mode == 'w':
                    dict_writer.writeheader()
                dict_writer.writerows(data_list)
                print(f"Successfully saved {len(data_list)} posts
to {OUTPUT_FILE}")
        except Exception as e:
            print(f"Error saving to CSV: {e}")

    def get_post_links(keyword, page_number):
        links = []
        params = {'q': keyword, 'page': page_number}
        try:
            time.sleep(DELAY_SECONDS)
            response = requests.get(SEARCH_URL, params=params,
headers=HEADERS, timeout=15)
            response.raise_for_status()
            soup = BeautifulSoup(response.text, 'html.parser')
            post_elements = soup.find_all('a',
href=re.compile(r'/topic/\d+'))
            for elem in post_elements:
                link = elem.get('href')

```



```

        if link:
            full_link = link if link.startswith('http')
else BASE_URL + link
            if full_link not in links:
                links.append(full_link)
except Exception as e:
    print(f"Error fetching search results (Page
{page_number}): {e}")
    return links

def scrape_post_content(post_url):
    try:
        time.sleep(DELAY_SECONDS)
        response = requests.get(post_url, headers=HEADERS,
timeout=15)
        response.raise_for_status()
        soup = BeautifulSoup(response.text, 'html.parser')
        title_elem = soup.find('h2', class_='display-post-
title')
        body_elem = soup.find('div', class_='display-post-
story')

        title = title_elem.get_text() if title_elem else ""
        body = body_elem.get_text() if body_elem else ""
        combined_text = f"{title}\n{body}"
        return clean_text(combined_text)
    except Exception as e:
        print(f"Error scraping post {post_url}: {e}")
        return None

def main():
    seen_ids = load_seen_ids()
    skipped_ids = set() # เก็บ ID ที่สั้นเกินไปในเซสชันนี้
    buffer = []
    total_scraped_this_run = 0
    current_total = len(seen_ids)

    print(f"Current dataset: {current_total} posts. Target:
{TOTAL_POST_LIMIT}")
    if current_total >= TOTAL_POST_LIMIT: return

    try:
        for keyword in KEYWORDS:
            if current_total >= TOTAL_POST_LIMIT: break
            print(f"\n>>> Keyword: '{keyword}'")

            for page in range(1, PAGES_PER_KEYWORD + 1):
                if current_total >= TOTAL_POST_LIMIT: break
                links = get_post_links(keyword, page)
                if not links: break

                new_links_found = False
                for link in links:
                    topic_id = extract_topic_id(link)

                    # ตรวจสอบว่าเคยดึงสำเร็จ หรือเคยข้ามไปแล้วหรือยัง
                    if not topic_id or topic_id in seen_ids
or topic_id in skipped_ids:

```

```

        continue

        new_links_found = True
        print(f"Scraping
[{current_total+1}/{TOTAL_POST_LIMIT}]: {link}")
        text = scrape_post_content(link)

        # ปรับเป็น > 200
        if text and len(text) > 200:
            buffer.append({'topic_id':
topic_id, 'text': text, 'label': 'Human'})
            seen_ids.add(topic_id)
            current_total += 1
            total_scraped_this_run += 1
            if len(buffer) >= SAVE_BATCH_SIZE:
                save_to_csv(buffer)
                buffer = []
        else:
            print(f"Skipping (Content too
short).")
            skipped_ids.add(topic_id)

        if not new_links_found:
            print(f"No new topics on page {page}.
Moving to next keyword.")
            break

    except KeyboardInterrupt:
        print("\nInterrupted. Saving...")
    finally:
        if buffer: save_to_csv(buffer)
        print(f"\nDone. Added {total_scraped_this_run}
posts. Final: {len(seen_ids)}")

if __name__ == "__main__":
    main()

```

ก.2 สคริปต์ทำความสะอาดข้อมูล

สคริปต์ลบ AI-generated preamble ทั้งภาษาไทย (เช่น "สวัสดี gang!") และภาษาอังกฤษ (เช่น "Certainly!", "Sure!", "Here is...") ออกจากต้นข้อความ รวมถึงลบส่วน Translation ที่ Llama 3 สร้างขึ้นโดยอัตโนมัติ

```

import pandas as pd
import re

INPUT_CSV = "combined_dataset.csv"
OUTPUT_CSV = "cleaned_dataset.csv"
df = pd.read_csv(INPUT_CSV)
def clean_text(text: str) -> str:
    if not isinstance(text, str):
        return text

```

```

# Remove "สวัสดี gang!" variants at the start of text
# Covers: สวัสดี gang! / สวัสดีคะ gang! / สวัสดีค่ะ gang! / สวัสดีgang!
etc.
text = re.sub(
    r'^สวัสดี(?:คะ|ค่ะ|
ครบ)?\s*(?:gang|crew|everyone|friends|fans) [!\! ]\s*',
    '',
    text,
    flags=re.IGNORECASE,
)

# Remove English AI preambles at the start
text = re.sub(
    r'^(?:Here(?:\s+is|\s+are)\b[^\n]*[.\n])|'
    r'Certainly[!,]?\s*|'
    r'Sure[!,]?\s*|'
    r'Of\s+course[!,]?\s*|'
    r'Absolutely[!,]?\s*|'
    r'Below\s+(?:is|are)\b[^\n]*[.\n]|'
    r'As\s+requested[.,]?\s*|',
    '',
    text,
    flags=re.IGNORECASE,
)

# Remove translation sections: (Translation: ...) -
single or multi-line
text = re.sub(r'\n*(Translation:[^)]*\n)', '', text)

# Remove standalone Translation: header lines
text = re.sub(
    r'\n*(?:Translation|Thai Translation|English
Translation)\s*:\s*\n',
    '\n',
    text,
    flags=re.IGNORECASE,
)

# Strip leading/trailing whitespace
text = text.strip()

return text

before_count = len(df)
df['text'] = df['text'].apply(clean_text)

# Drop rows that became empty after cleaning
df = df[df['text'].str.strip().ne('')]
after_count = len(df)

df.to_csv(OUTPUT_CSV, index=False, encoding='utf-8-sig')

print(f"Input rows : {before_count}")
print(f"Output rows : {after_count}")
print(f"Dropped : {before_count - after_count}")
print(f"Saved to : {OUTPUT_CSV}")

```